

SynapseOS: Designing and Evaluating a Conversational Interface Layer for Personal Computing

by

Alexandra Sulit

Willard Soriano

Steven Evian Lozano

A Thesis Proposal Submitted to the Department of Computer Science
in Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science

Mapúa University – Makati
September 2026

APPROVAL PAGE

[To be inserted upon approval.]

ACKNOWLEDGEMENT

[To be inserted prior to final submission.]

TABLE OF CONTENTS

TITLE PAGE	1
APPROVAL PAGE	2
ACKNOWLEDGEMENT	3
TABLE OF CONTENTS	4
LIST OF TABLES	6
LIST OF FIGURES	7
ABSTRACT	8
Chapter 1: INTRODUCTION	9
Gap / Opportunity	10
Problem Statement	11
Objectives	12
Contributions	12
Significance of the Study	14
1.1 Scope and Limitations of the Study	14
Notes	15
Chapter 2: REVIEW OF RELATED LITERATURE	17
2.1 Introduction to the Review	16
2.2 Theoretical Framework	16
2.3 Historical Foundations of Natural-Language Interfaces	18
2.4 Natural-Language-to-Shell Translation	20
2.5 Graphical and Computer-Using Agents	21
2.6 Accessibility and Speech-Driven Interaction	24
2.7 Evaluation Environments and Benchmarks	25
2.8 Synthesis: Comparative Analysis and Research Gap	25
2.9 Review of the Methodological Literature	30
2.10 Summary	31
Chapter 3: METHODOLOGY	36
Conceptual Framework	34
System Environment Specifications	35
Section 1: Dataset	37
Section 2: Procedure / Experiment	42
Section 3: System Testing	49

REFERENCES**65****APPENDICES****70**

LIST OF TABLES

Table 1.1: Components adopted rather than built, and why	13
Table 2.1: Comparative analysis of surveyed systems along the four-layer pipeline	29
Table 3.1: Study evaluation machines	38
Table 3.2: SynapseOS minimum deployment requirements	39
Table 3.3: Metrics for the recoverability algorithm, and what each is for	57
Table C.1: Custom cross-platform task suite — core tasks	72
Table C.2: Boundary tasks — outside shell expressibility	73

LIST OF FIGURES

Figure 2.1: The four-layer interaction pipeline for language-mediated computing interfaces	18
Figure 3.1: Conceptual framework of SynapseOS	37

ABSTRACT

A computer can do far more than most people using it can reach. The command line expresses nearly any operation a system supports, but only for someone who can write a scripting language such as bash or PowerShell. This study designs, implements, and evaluates SynapseOS, a conversational interface layer in which the user states an intention in ordinary language and the system carries it out through shell commands on Linux, with inference running locally. Letting a language model act on a real system raises a problem the interface cannot solve: a generated command may destroy data before its author can read it, and existing safeguards judge a command by its name rather than by what it does. The principal technical contribution is an algorithm that finds what a generated command will change before it runs. It sees through the wrappers a line hides its effects behind — *find -exec*, *xargs*, loops, and command substitution — and resolves targets computed at run time with read-only dry runs, so that exactly what would be lost can be captured and the command undone. It is evaluated on a labelled corpus against the pattern matching it extends and against a list that refuses everything it does not recognise, reporting commands that lose data silently and the share of captures that restore the prior state. SynapseOS is evaluated separately, in a within-subjects study against each participant's own operating system. The two evaluations are independent.

Keywords: conversational interface, human-computer interaction, local language model, Linux desktop, user study

Chapter 1

INTRODUCTION

Ask a person to gather every photo they took last June into a single folder and they will do it — opening a file browser, sorting by date, selecting, dragging. Ask a computer the same thing, in those same words, and for fifty years it could not. The instruction has always had to be translated first into the machine’s own vocabulary: a path, a menu, a sequence of clicks, or a command typed with exact syntax. The graphical desktop made that translation visual rather than textual, but it did not remove it. The person still adapts to the machine. This study asks whether that is now optional.

The dominant paradigm for personal computing has remained largely unchanged since the early 1980s, when the graphical desktop — icons, menus, and pointer-driven interaction — was established through the Xerox Star and Apple Lisa lineage. Despite its longevity, a parallel research tradition has long explored whether users should instead be able to describe what they want in natural language and have the computer carry it out. The earliest serious system in this tradition, the Berkeley UNIX Consultant, was already operational by the late 1980s [1]. Decades later, the rise of large language models (LLMs) and multimodal foundation models¹ has reopened this thread with substantially greater technical capability.

The field that has emerged sits at the intersection of human-computer interaction, operating systems, natural language processing, and computer vision. On the shell side, recent work has demonstrated that LLMs can translate natural language into Bash commands with usable accuracy and can be embedded as a Unix primitive with appropriate safety guardrails [2, 3]. On the graphical side, so-called computer-using agents² now operate desktop and mobile applications by combining native APIs with vision-based grounding [16, 18]. On the web, language-guided agents complete tasks across realistic websites with measurable but limited reliability [8, 9]. A converging research thread approaches the same technical problem from an accessibility perspective: speech-driven GUI command mapping for users with motor and speech impairments has progressed from voice-to-text dictation toward voice-to-action agents, with recent work on speech-instructed GUI agents [21] and region-aware visual grounding [22] demonstrating that the underlying components are maturing rapidly. Yet the gap between demonstration and deployable replacement remains substantial. Capability on short, self-contained tasks has advanced quickly, but sustained computer work has not: on OSWorld 2.0, a benchmark of long-horizon tasks whose median item takes a skilled human roughly an hour and a half, the best frontier configuration completes only 20.6% of tasks outright [26].

This rapid progress has been uneven across platforms. Windows is heavily represented in the desktop agent literature [15, 16], Android in the mobile agent literature [10, 11, 12], and the browser in the web agent literature [7, 8, 9]. Linux desktop coverage, by contrast, is conspicuously absent: existing work either addresses Linux only at the shell level [3] or treats it as one of several evaluation platforms rather than a primary target [17]. No surveyed system implements a working conversational session layer³ for the Linux desktop, despite Linux serving as the substrate for most developer and server-side workflows.

Gap / Opportunity

Three compounding gaps in the existing literature motivate the present research. First, no *implemented and empirically evaluated* system replaces the conventional desktop session layer with a conversational one. The idea is no longer unclaimed — AgentOS [28] proposes exactly such a natural-language portal in place of the graphical desktop — but it is offered as a research direction, without a built system or an evaluation. The systems that are built automate on top of the desktop rather than replacing it: UFO², for instance, positions itself as a "Desktop AgentOS" but operates as an automation layer atop the existing Windows shell, deliberately confining itself to a separate desktop so that agent and user do not interfere [16]. Second, the Linux work that exists serves servers and agents rather than the desktop session of an end user. NaSh addresses Linux at the shell level as a design proposal without evaluation [3]; shipping natural-language shells such as ANOLISA pair intent translation with approval gating and rollback, but scope themselves explicitly to server and DevOps workloads [29]; and OSWorld includes Ubuntu as an evaluation platform rather than a deployment target [17]. Third, controlled comparison against the conventional graphical workflow exists only for narrow slices of the problem. AXIS evaluates an API-first agent against manual operation within a single application, Microsoft Word, using a cloud-hosted frontier model [25], and the accessibility literature evaluates assistance for specific populations [5]. No study compares a conversational interface against the conventional workflow across operating-system-level tasks, on a Linux desktop, driven by a locally-hosted small model, under an explicit safety gate.

Notably, the accessibility-focused thread of the field exhibits the inverse limitation. Speech-to-GUI mapping research targets specific user populations — users with motor impairments, speech disorders, or cognitive disabilities — with specialized techniques such as speech-instructed GUI agents and resilient visual grounding [21, 22], but operates at the level of individual applications or interaction techniques rather than at the level of the session layer itself. These approaches presuppose an underlying conversational system surface on which to run, yet no such surface exists. This complementarity sharpens the opportunity: a general-purpose conversational interface layer would serve not only mainstream users but also provide the substrate that accessibility-specific configurations require.

These gaps present a clear opportunity, and a precedent for how to seize it. The history of the Unix shell demonstrates that a new interactive layer — `bash` succeeding the Bourne shell, `zsh` and `fish` succeeding both — can replace the software a user directly interacts with while leaving the kernel and the core system utilities entirely unchanged, producing a materially different experience for the user without touching anything beneath the session. The opportunity addressed by this study is to apply the same pattern in a new direction, and with one deliberate departure from it. SynapseOS adds a conversational interface layer to a Linux desktop that continues running underneath it: the desktop shell, window manager, and application launcher are left in place, as are the kernel, the core system utilities, and the existing application ecosystem. A user retains every route to the machine they already had and gains a third. The departure from the shell precedent is that `zsh` displaced `bash` for those who adopted it, whereas this layer displaces nothing — a conversational interface cannot yet perform the visual tasks a graphical desktop performs, so replacing the desktop would remove capability from the user in exchange for convenience, and adding to it costs them nothing. It is designed to serve as the primary user-facing interface to a personal computing environment — spanning novice, elderly, non-technical, and technical user populations — with an evaluation methodology that engages the conventional desktop workflow as its baseline rather than as its substrate.

Problem Statement

The capability of a personal computer is unevenly reachable. Nearly any operation a system supports can be expressed on its command line, but only by someone who can write in a shell scripting language; everyone else is limited to the operations a graphical desktop was built to expose. This study addresses the absence of a conversational interface layer — one that accepts intent in ordinary language and carries it out through the standard system toolchain, replacing the desktop's graphical session in the way a new interactive shell replaces the one before it rather than modifying the kernel or core utilities — that has been empirically evaluated against the conventional graphical workflow it would displace, and evaluated specifically for whether it changes what people of differing technical fluency can accomplish.

Specifically, the study seeks to answer the following questions:

1. Can the effects of a generated shell command be found before it runs — seeing through the wrappers that hide its targets and resolving targets computed at run time — so that the minimal set of pre-images sufficient to restore the prior state can be captured, losing less data silently and asking the user less often than matching the command against an enumerated list of destructive patterns, and than a list that refuses every command it does not recognise?
2. How can a conversational interface layer be designed so that a person can direct an operating system in ordinary language, reaching the full range of operations the command line can express without knowing the syntax that command line requires?

3. What confirmation and recovery design allows a user to act on their own system through natural language while preserving their ability to undo what was done — including when a command was approved in error?
4. Does a conversational interface narrow the gap between what technically fluent users and users fluent in neither the command line nor the graphical desktop can accomplish on a computer, and how does the resulting experience — task success, time, error rate, and cognitive workload — compare to the interface each participant uses today?

Objectives

The general objective of this study is to design, implement, and evaluate SynapseOS, a conversational interface layer that lets a person operate a computer in ordinary language rather than through memorized command syntax or the bounded set of actions a graphical desktop exposes. It is realized as a replacement for the conventional graphical session layer — the desktop shell, session manager, and application launcher — while leaving the underlying Linux kernel, core system utilities, and application ecosystem unchanged.

Specifically, this study aims:

1. To develop and evaluate an algorithm that derives, from a generated command's syntactic structure rather than its name, the set of system effects it would produce, resolving through wrappers and run-time targets, and computes from it a minimal capture plan; and to measure it on a labelled command corpus against two baselines — the enumerated-pattern list it extends and a list that fails closed — reporting silent loss separately, since a command that loses data unasked and uncaptured is the failure that destroys a user's data, and reporting how often a capture actually restores the prior state.
2. To design a conversational interface layer for the Linux desktop that exposes the full range of shell-expressible operations through ordinary language, requiring no knowledge of shell syntax from the person using it.
3. To develop a natural-language-to-bash intent parsing and execution pipeline for SynapseOS, incorporating a reversibility-based confirmation gate and an undo mechanism, balancing intent parsing accuracy against execution safety.
4. To implement SynapseOS as a Linux session manager⁴ that provides a persistent conversational surface, a structured undo log, and a confirmation policy for irreversible operations, addressing the safety and recoverability gaps identified in prior work [3, 16].
5. To evaluate SynapseOS through a controlled user study against each participant's own primary operating system, measuring task success, completion time, error rate, and cognitive workload across a population fluent in neither existing interface and a population fluent in both — with the

difference between those two populations as the primary outcome of interest, since the central claim of this work is that a conversational interface narrows the gap between them.

Contributions

This study's design principle is to adopt existing software wherever it will serve and to build only what cannot be adopted. The principle is stated here because it is what makes the contribution identifiable: if everything that can be taken off the shelf is taken off the shelf, then whatever remains is the original work by construction rather than by assertion. Table 1.1 states what is adopted, so that what follows it can be read against a complete account of what is not.

Table 1.1. Components adopted rather than built, and why.

Concern	Adopted	Why it is not built here
Kernel, init, system services, userland	Debian 13 (Trixie)	Writing these is operating-system development in the strict sense and is explicitly outside this study
Desktop environment, window manager, display server	XFCE on X11	The interface layer runs over the desktop and leaves it working; reimplementing it would remove capability from the user
Model inference and serving	Ollama	Model lifecycle, quantised loading, and request serving are solved and are not the research question
Language model	Qwen2.5-Coder-3B-Instruct	Training a model is a different study; this one evaluates an interface built on an existing one
Command execution	The Linux shell and core utilities	Driving the existing toolchain is the system's premise, not a limitation of it
Version-control recovery	Git's object store	A captured commit identifier restores prior state more reliably and far more cheaply than any file-level copy

Against that, the study contributes the following.

1. **An algorithm for recoverability analysis of generated shell commands.** Given a command produced by a language model, the algorithm finds what it would change and, where that can be reversed only with preparation, computes the minimal set of pre-images sufficient to restore the prior state. It makes two claims and no others: it sees through the wrappers a line hides its effects behind, and it resolves targets computed at run time before the command runs. Deciding whether a command is dangerous is not among them: a list that refuses every command it does not recognise reaches the same verdict on the commands studied here, and the contribution is the capture that follows. Its formal basis is an *effect semantics*: a command is represented not by its name but by the set of system effects its syntactic structure implies — which paths it would read, create, write, remove, or alter in metadata — resolved against a working directory and propagated through the operators that join commands into one line. The recovery plan is a minimum-cost cover of that

effect set, and the verdict a predicate over it. This is the study's principal technical contribution and the subject of RQ1. It is distinguished from the permission models used by existing agentic tools, which decide whether a command is allowed by matching it against enumerated patterns, and which take no pre-images: a permission model can refuse a command, but it cannot restore what an approved command destroyed. It is equally distinguished from privilege mechanisms such as *sudo* and *polkit*, which gate on who is acting rather than on what is about to happen. The algorithmic difficulty lies in composition and in targets that exist only while the command runs — a single generated line may chain several operations through *&&*, pipelines, subshells, and redirects, and *rm \$(ls *.log)* or *find -delete* names nothing until it executes — which is what an enumerated pattern list cannot express. Soundness is deliberately one-sided: anything the analysis cannot resolve is treated as unrecoverable, because over-approximating destructiveness costs a confirmation prompt whereas under-approximating it destroys data. The analysis is not free of lists: it rests on a table of what each known command does, and a command absent from that table is unknown and asks. Its claim is what it adds on top of a table, not that it has none.

2. **A working conversational interface layer for a Linux desktop** that grounds natural-language intent in shell commands, gates irreversible operations on explicit confirmation, and records a recovery journal, implemented as a single binary running over an unmodified desktop session.
3. **A controlled comparative evaluation of that interface against the one each participant already uses**, on matched tasks, with the difference between users fluent in neither existing interface and users fluent in both as the primary outcome. This addresses a gap in the literature identified in Chapter 2, where conversational computing interfaces are benchmarked as autonomous agents but seldom measured against the conventional workflow they would supplement.

The first contribution is evaluated against a labelled command corpus and is therefore independent of the user study; the third is evaluated through the study itself. They are reported separately for that reason, and a negative result on either is reported as such.

Significance of the Study

The findings of this study are expected to benefit the following groups:

1. **Novice and non-technical computer users.** The conventional graphical desktop imposes a steep learning curve rooted in its icon-and-menu paradigm. A conversational interface layer that accepts natural language intent lowers the threshold for productive computer use, particularly for users with limited prior computing experience, including elderly users encountering desktop computing late or infrequently.

2. **Developers and power users on Linux.** A conversational interface layer reduces the cognitive load of recalling and composing shell commands by exposing system operations through a unified natural-language interface.
3. **Users with motor accessibility needs.** Although this population is scoped out of the initial evaluation for ethical and recruitment reasons, SynapseOS is positioned to serve as the session-level substrate on which accessibility-specific configurations can operate. Prior work has established design guidelines for LLM-based speech interfaces for users with motor impairments [5] and has demonstrated specialized techniques — speech-instructed GUI agents and resilient visual grounding — at the application and interaction-technique level [21, 22]. These techniques presuppose a conversational system surface that does not yet exist at the session level; SynapseOS provides that surface, making accessibility-specific extensions a natural and well-grounded future direction rather than a speculative one.
4. **The research community.** The controlled comparative evaluation methodology this study introduces addresses a recognized gap in the literature, where conversational interfaces are rarely compared against conventional graphical workflows on the same tasks and population [17]. The evaluation design provides a replicable methodology for future work in this space.

1.1 Scope and Limitations of the Study

Two terms are used deliberately throughout and are not interchangeable. *Conversational interface layer* names what the user experiences: the surface through which intent is expressed and results are returned. *Session layer* names where that surface sits in the system: the software a user meets immediately after login. The first is this study's contribution; the second is its architectural position, and the phrase “session layer” is used only when what SynapseOS displaces is specifically at issue. This study covers the design, implementation, and evaluation of SynapseOS as a conversational interface layer for the Linux desktop. SynapseOS runs as an agentic layer over an ordinary desktop session rather than in place of one: the desktop shell, session manager, and application launcher continue to run and remain available to the user, as do the kernel and the core system utilities beneath them. The Linux kernel, the core system utilities, and the existing ecosystem of Linux applications are not modified by this study; SynapseOS interacts with the system by translating natural language into shell commands executed through the standard command-line toolchain, rather than by rewriting applications or the utilities they depend on. The subject matter spans natural language processing, LLM-based reasoning, command-line automation, Linux system integration, and human-computer interaction.

The study examines cross-application desktop workflows on Linux, including shell operations, file management, and application-level tasks representable in the OSWorld benchmark task suite [17]. The population studied consists of novice computer users and power users recruited for a controlled user study.

The study is conducted in a Linux desktop environment and is bounded by the duration of the thesis project.

The following limitations are acknowledged. First, SynapseOS commits to a locally-hosted, code-specialized small language model (Qwen2.5-Coder-3B-Instruct) rather than a hosted frontier model, prioritizing privacy and offline operation over the potentially higher capability of larger hosted alternatives⁵; this tradeoff may affect the ceiling on intent parsing accuracy achievable within the study. Second, users with motor accessibility needs — a highly relevant population — are excluded from the initial evaluation due to ethics-review and recruitment constraints; accessibility-specific configurations such as speech-instructed agent operation for atypical speech patterns [21] are likewise out of scope for the initial implementation but represent a natural extension of the SynapseOS substrate. Third, SynapseOS targets the Linux desktop specifically, and the findings may not generalize directly to Windows or macOS desktop environments. Fourth, the safety confirmation policy for irreversible system operations is defined in principle but requires empirical calibration during implementation. Fifth, evaluation is bounded by the task coverage of the OSWorld benchmark and the custom task suite developed for this study; tasks outside this coverage are not evaluated.

Notes

1. Foundation models are large neural networks trained on broad, general-purpose data and adaptable to many tasks. Multimodal variants accept multiple input types — typically text and images — enabling the model to interpret visual interfaces and screenshots alongside natural language.
2. Computer-using agents are AI systems that interact with a computer's graphical environment — clicking, typing, reading the screen — to complete tasks on behalf of a user. Unlike shell-level automation, they operate through the same visual interface a human would use.
3. The session layer (or simply, the session) is the software the user directly interacts with immediately after login — the desktop shell, window manager, session manager, and application launcher on a graphical system, or the login shell on a text terminal. It is distinct from the userland (or user space): the core system libraries and command-line tools (e.g., glibc, coreutils, bash, apt) that run outside the kernel and that the session layer itself depends on and invokes. SynapseOS replaces the session layer; the underlying userland is unchanged.
4. A Linux session manager is the program that starts after user login and establishes the graphical environment — launching the window manager, desktop shell, and background services. SynapseOS replaces this component with a conversational interface, making it the first program a user encounters after login.
5. Open-weights models make their trained parameters publicly available for local download and use. Users run inference on their own hardware without routing queries to an external service provider, enabling offline operation and preventing user data from being transmitted to third parties.

Chapter 2

REVIEW OF RELATED LITERATURE

2.1 Introduction to the Review

This chapter reviews the body of research relevant to the design and evaluation of a conversational interface layer for the Linux desktop. The review spans nearly four decades of work on natural-language interfaces to computing systems, from the symbolic-artificial-intelligence consultants developed for the Unix shell in the late 1980s [1] to the large language model (LLM) agents that now operate shells, graphical applications, browsers, and mobile devices. Its purpose is threefold: to establish the conceptual vocabulary and technical foundations on which the present study builds, to characterize the current state of the field and its recurring limitations, and to locate precisely the gap that the proposed system, SynapseOS, is designed to fill.

2.2 Theoretical Framework

The heterogeneous systems surveyed in this chapter — shell translators, web and mobile agents, desktop automation frameworks, and accessibility interfaces — differ widely in platform, action space, and evaluation method, yet they address a common problem: converting a user's expressed intent into effects on a computing system. To compare them on common ground, this review adopts a *four-layer interaction pipeline* — Input, Reasoning, Action, and Operating-System Integration — as its theoretical framework. The framework is adapted from the layered decomposition of large language model (LLM)-brained graphical user interface (GUI) agents proposed in the survey of Zhang and colleagues [18], generalized here from graphical agents specifically to language-mediated computing interfaces as a class. Figure 2.1 depicts the pipeline and situates the layers relative to one another.

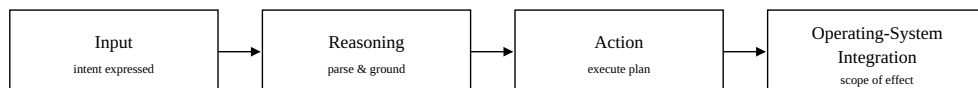


Figure 2.1. The four-layer interaction pipeline for language-mediated computing interfaces, adapted from the LLM-brained GUI-agent survey of Zhang and colleagues [18]. A user's intent enters at the Input layer, is parsed and grounded at the Reasoning layer, executed at the Action layer, and takes effect within a scope defined at the Operating-System Integration layer.

The *Input* layer concerns how a user expresses intent, whether by typed text, speech, or structured shortcuts. The *Reasoning* layer concerns how that intent is parsed into a plan and grounded into the affordances of the environment. The *Action* layer concerns how a grounded plan is executed — through command translation, through binding to native application programming interfaces (APIs), or through simulated keyboard and mouse operations. The *Operating-System Integration* layer concerns the scope of computing an approach can affect, from a single application to an entire operating system.

This framework is appropriate to the present study for three reasons. First, it is *modality-neutral*: because it abstracts over whether intent arrives as text, speech, or gesture and whether action is realized through shell commands, API calls, or simulated clicks, it accommodates the full span of surveyed systems without privileging any one interaction style — a necessary property for a review that spans command-line, graphical, and accessibility work. Second, it is *diagnostic*: locating each system's principal limitation at a specific layer — visual grounding as a Reasoning-and-Action failure, or narrow platform coverage as an Operating-System Integration limitation — turns an otherwise flat catalogue of systems into a structured comparison, and it is this layer-by-layer positioning that the comparative synthesis of §2.8 makes explicit. Third, it is *directly applicable* to the proposed system: the conceptual framework of SynapseOS presented in Chapter 3 instantiates this same pipeline — accepting natural-language intent at the Input layer, parsing it with a locally-hosted small language model at the Reasoning layer, translating it into shell commands executed through the standard toolchain at the Action layer, and spanning the full graphical session rather than a single application at the Operating-System Integration layer. The framework therefore serves both as the organizing lens for the literature and as the vocabulary in which the study's own contribution is later positioned.

Works were included in this review if they satisfy at least one of five criteria: they translate natural language into executable system actions; they operate graphical, web, or mobile interfaces through LLM-based reasoning; they address speech- or accessibility-driven interaction with computing systems; they provide an evaluation environment for such systems; or they report a controlled empirical comparison between a language-mediated interface and a conventional graphical one. The fifth criterion is stated explicitly because the third gap this chapter identifies concerns the scarcity of exactly such comparisons: a corpus admitting only the first four criteria could not contain a comparative interface study by construction, and an absence argument drawn from it would be circular rather than evidential. Works concerned solely

with model architecture or training, without a computing-interaction target, and general-purpose conversational systems without a system-control capability, were excluded as outside the scope of this study.

Candidate works were identified through language-model-assisted search across arXiv, the ACM Digital Library, IEEE Xplore, and the ACL Anthology, extended by backward and forward citation chaining from each work that met the criteria above. This review is therefore a narrative review rather than a systematic one, and the search is not claimed to be exhaustive. Because assisted search can surface plausible but non-existent references, and can attribute to a source a claim that source does not make, every work cited in this chapter was subsequently verified by hand against its primary record — author list, venue, and year — and, where a specific claim is attributed to a work, against the presence of that claim in the work's own text. That verification pass corrected errors of both attribution and currency introduced by the initial search, and it is reported here because the reliability of this chapter rests on it rather than on the discovery method. The limitation this leaves is one of recall rather than accuracy: assisted search favours well-indexed and heavily cited work, so the corpus may under-represent less visible literature. The absence claims in §2.8 are accordingly scoped to the surveyed corpus and stated as such, rather than as claims about the literature as a whole. The chapter first sets out the theoretical framework that organizes the review (§2.2), then proceeds thematically — historical foundations (§2.3), the shell and command-line thread most directly related to the proposed design (§2.4), graphical and computer-using agents (§2.5), accessibility and speech-driven interaction (§2.6), and evaluation environments (§2.7) — before synthesizing the surveyed works into a comparative analysis and an explicit statement of the research gap (§2.8). It closes with a review of the methodological literature that informs the study's evaluation design (§2.9) and a summary that motivates the methodology detailed in Chapter 3 (§2.10).

2.3 Historical Foundations of Natural-Language Interfaces

The aspiration to let users describe what they want in ordinary language and have a computer carry it out is not new. The earliest serious system in this tradition, the Berkeley Unix Consultant of Wilensky and colleagues [1], was already operational by the late 1980s. The Unix Consultant was an intelligent natural-language interface that allowed naive users to learn about the Unix operating system through conversation in ordinary English, addressing both the natural-language-frontend problem and the consultation problem, and thereby distinguishing itself from systems intended only as command translators. It was built on the KODIAK knowledge representation and incorporated a user-modeling component that represented the user's knowledge state and a teaching component through which users could extend its knowledge base in natural language. The system demonstrated end-to-end natural-language consultation about Unix, successfully answering "how do I," definitional, and problem-solving questions. Critically, its

authors identified knowledge acquisition as the principal bottleneck: extending the system to a new domain required substantial hand-engineering — a concern that directly foreshadows the contemporary problem of LLM-based agents requiring per-application grounding data.

The intervening decades saw natural-language interaction treated primarily as a supervised-learning problem. Representative of this pre-LLM era, the work of Li and colleagues on mapping natural-language instructions to mobile user-interface action sequences [19] trained dedicated phrase-extraction and grounding transformers on hand-annotated action sequences, achieving 70.59% accuracy on predicting complete ground-truth action sequences on its benchmark but remaining bound to its purpose-built datasets and architectures. The decisive shift came with the introduction of in-context reasoning patterns for LLMs. ReAct, introduced by Yao and colleagues [20], collapsed the multi-stage supervised pipeline into a prompting strategy: the language model is prompted to alternate between free-text "Thought" steps that update its plan and "Action" steps that interact with an external environment, with observations fed back into subsequent reasoning. Implemented purely as a prompting strategy without fine-tuning, ReAct nonetheless outperformed imitation-learning and reinforcement-learning baselines that required thousands of training trajectories.

The historical significance of this transition is twofold, and it frames the remainder of this review. First, it explains why progress in the field has been so rapid since 2023: the methodological foundations became portable across modalities and platforms in a way that the earlier dataset-bound approaches were not. Nearly every system surveyed in the sections that follow inherits from ReAct the convention of interleaving reasoning traces with environment-affecting actions. Second, this shared inheritance underscores a corresponding fragility: these systems inherit the failure modes of LLM reasoning — hallucinated actions, plan inconsistency, and brittleness on long-horizon tasks — alongside their capabilities, a concern that the evaluation literature reviewed in §2.7 quantifies.

Interface Paradigms and the Case Against Natural Language

The aspiration to a natural-language interface has a long-standing critique that remains unanswered, and a review that omitted it would misrepresent the state of the question. Shneiderman's account of direct manipulation [31] holds that interfaces succeed when the objects of interest are continuously visible, when actions upon them are rapid, incremental, and reversible, and when their effects are immediately apparent — properties that let a user build and maintain an accurate model of system state. Norman formalized what an interface does for a user as the bridging of two gulfs [32]: the gulf of execution, between an intention and the actions a system makes available, and the gulf of evaluation, between the system's response and the user's ability to interpret it. Hutchins, Hollan, and Norman [33] argued that direct manipulation narrows both, by making an interface feel like the object of interest itself rather than a description of it.

A language interface sits awkwardly against nearly every one of those properties. An utterance is not continuously visible, not incremental, and not reversible by the same gesture that produced it; the set of available actions is not displayed, so the user must guess at the system's capabilities rather than see them. Shneiderman and Maes debated precisely this trade-off between direct manipulation and interface agents [34], and Shneiderman argued subsequently that speech and language interaction consume the same cognitive resources a user needs in order to hold a task in mind, where pointing does not [35]. Empirical work on modern conversational agents has largely borne the concern out: Luger and Sellen documented a persistent gulf between what users expected a conversational agent to understand and what it actually did [36], and Clark and colleagues found that the properties people value in human conversation transfer poorly to interaction with an agent [37].

This critique is the strongest argument against the premise of the present study, and it shapes the system's design rather than being set aside by it. Two of its three concerns are addressed directly. The gulf of evaluation is narrowed by displaying the generated command and its raw output rather than reporting an outcome in prose, so that the user sees what was actually done rather than a description of it. The irreversibility of an utterance is answered by the confirmation gate and the undo journal, which restore by mechanism the reversibility that direct manipulation obtains for free. The third concern — that a user cannot see the boundary of what the system is able to do — is not solved by this design and is carried forward as an acknowledged limitation.

The comparative question is older than the current literature and considerably less settled than it is often assumed to be. Norman's critique of the UNIX command line [38] remains the standard reference for the position that the command line's power is inaccessible to most people in practice. But the best-known controlled comparison of command, menu, and iconic interfaces, conducted by Whiteside and colleagues across seventy-six users and seven systems [39], concluded that interface style itself did not predict either performance or preference: careful design did, and the system that served novices best also served experts best. That finding is as much a caution for the present study as a motivation for it. It warns against attributing to a conversational paradigm what may in fact belong to the quality of one particular implementation, and it is a principal reason this study measures a specific system against a specific baseline rather than claiming to measure one paradigm against another.

2.4 Natural-Language-to-Shell Translation

The thread of the literature most directly related to the proposed system concerns the translation of natural language into shell commands. Because SynapseOS grounds user intent into commands executed through the standard Linux command-line toolchain, the accuracy, safety, and context-awareness of natural-language-to-Bash translation are of central relevance to its design.

Westenfelder and colleagues [2] address this translation problem directly. Their work constructs a manually verified test set of 600 instruction–command pairs and a training set of 40,939 pairs, and introduces a functional-equivalence heuristic that reaches 95% confidence in determining whether two Bash commands produce the same effect — allowing automated evaluation to move beyond exact-string matching to a measure of whether a generated command actually accomplishes the intended task. Their fine-tuned models substantially outperform zero-shot baselines, demonstrating that fine-tuning specifically on command-translation data is effective. The authors identify two persistent limitations: ambiguity in user instructions, and the absence of system context — the contents of the current directory, the user's command history, and environment variables — at translation time. Integrating context-aware retrieval is identified as a direction, one that the persistent conversational session of SynapseOS is well positioned to supply.

Where translation accuracy is the concern of Westenfelder and colleagues, safety is the concern of Gyawali and colleagues, whose NaSh system [3] proposes guardrails for an LLM-powered natural-language shell for Linux. Published under the operating-systems classification, NaSh interposes a guardrail layer between the user's natural-language input and the LLM-generated command: the model proposes a candidate command, the guardrail layer evaluates it against safety, reversibility, and confirmation criteria, and the user receives a structured preview before execution. The authors note that LLM outputs may be unintended or unexplainable, requiring guardrails for user error recovery, and they leave as open work both the design of recovery and rollback mechanisms for irreversible system operations and the empirical user evaluation of the approach. These two open problems — reversibility-aware confirmation and empirical validation with users — are precisely the concerns that the confirmation gate, undo mechanism, and controlled user study of the present study are designed to address.

The feasibility of deploying such a system without reliance on a hosted frontier model is supported by the large-scale evaluation of Jacobs and colleagues [24], who assess a broad range of locally deployable open-source LLMs on Bash command generation. Their work provides an empirical anchor for the viability of running natural-language-to-Bash translation on local, open-weights models of modest size — the deployment posture adopted by SynapseOS for reasons of privacy and offline operation. Taken together, the shell-and-command-line thread establishes that natural-language-to-Bash translation is tractable, that safety and reversibility are central rather than incidental concerns, and that local deployment is feasible; yet no surveyed shell-level system has been evaluated with users against the conventional workflow it would supplement.

One gap within that thread is worth isolating, because it is the one this study's algorithmic contribution addresses. Every surveyed system that guards a generated command does so by *permission*: the command is matched against an enumerated set of patterns judged dangerous, and is then allowed, refused, or escalated to the user. NaSh [3] proposes guardrails of this kind and, notably, identifies selective undo as a requirement while leaving its mechanism open; the agentic coding assistants discussed in §2.8

operate the same way in practice, with allow-and-deny lists over command prefixes. A permission model answers whether a command may run. It does not answer whether, having run, the command can be undone — and those are different questions with different answers. Refusing a command protects a user only until they approve it, which for any system that must let work proceed is most of the time. The distinction matters because enumeration does not compose. A pattern list matches a command in isolation, whereas a generated line routinely chains several operations whose recovery requirements differ — a removal, a version-control reset, and an in-place edit may appear in one invocation joined by `&&`, with redirects and command substitution altering what each actually touches. No enumeration of command prefixes expresses that, and extending the list does not converge: each newly named tool leaves the next one unnamed. Determining recoverability over a composed command, and computing the minimal information needed to reverse it, is the open problem this study takes up (RQ1), and it is what separates the present work from the guardrail designs surveyed above. The comparison that matters is not only against an enumerated list of dangerous commands. A list can be turned around so that it refuses every command it does not recognise, which removes the first weakness above at the cost of asking about everything unfamiliar; that fail-closed list is the strongest simple baseline, and the study therefore claims what the analysis adds to it — seeing through wrappers, resolving run-time targets, and capturing exactly what would be lost — and not the verdict alone.

The technique this recoverability judgment relies on — determining what a piece of code will do without executing it — is not new; it has a decades-long history in programming-language theory under two related names, neither of which the systems surveyed above draw on. Abstract interpretation, introduced by Cousot and Cousot [45], is the foundational technique for computing sound, conservative approximations of a program's runtime behaviour by evaluating it over an abstract domain rather than its concrete one. Type-and-effect systems, developed by Jouvelot and Gifford [46] and given a full formal treatment by Talpin and Jouvelot [47], extend a program's type judgement with an explicit account of the side effects an expression may produce, so that a compiler or static checker can reason about those effects instead of leaving them implicit in execution. The recoverability algorithm this study contributes (§2.1(c)) is an instance of this discipline applied to a domain neither tradition has targeted: a generated shell command is treated as an expression whose effect — on the filesystem, rather than on a type system's abstract state — is derived from its syntax and composed across its operators, and it is that effect set, not the command's name, that the recoverability verdict is computed over. This connects the algorithm to an established formal lineage rather than presenting it as a bespoke heuristic, and it grounds the algorithm's one-sided soundness property — over-approximating destructiveness under uncertainty rather than under-approximating it — in the same requirement abstract interpretation itself imposes: a sound abstraction may lose precision, but it may never assert a property the concrete execution does not actually have.

Static analysis of shell scripts specifically is comparatively little studied, and the existing work targets properties adjacent to recoverability rather than recoverability itself. ABash [48] performs abstract interpretation over bash's variable-expansion semantics to catch common expansion bugs and includes taint tracking, but is not concerned with what a script's commands would do to the filesystem if executed correctly. Smoosh [49] gives the POSIX shell a mechanized, executable small-step operational semantics, validated against other shells' conformance test suites — exactly the kind of ground-truth semantics a static effect analysis would need to be checked against — but stops at defining that semantics rather than analysing programs written in it. Closest of the three is CoLiS [50], which applies symbolic execution to the more than twenty-seven thousand POSIX-shell maintainer scripts in the Debian package archive to verify *idempotence* — that running a script a second time causes no further harm — a property Debian policy requires of these scripts because they run with root privileges during install, upgrade, and removal. Idempotence and recoverability are related but distinct: a script can be idempotent while still being destructive on its first run — `rm -rf /var/cache/pkg` is idempotent, since running it twice does no more damage than once, and unrecoverable, since the first run already destroyed what a recovery would need — and a command can be recoverable without being idempotent, if each run produces a new, independently-reversible effect. CoLiS is therefore the nearest prior application of static effect reasoning to shell scripts specifically, and it stops short of the present study's question for exactly this reason: verifying that re-execution is safe does not determine whether the first execution can be undone. No work identified in this review derives a recoverability verdict for a shell command from its statically-computed effects; that is the specific gap the present study's algorithmic contribution fills.

2.5 Graphical and Computer-Using Agents

A parallel and larger body of work addresses computer-using agents¹ — systems that operate graphical environments by issuing actions and observing results. Although SynapseOS operates at the command line rather than by simulating graphical interaction, this literature is essential context: it establishes the dominant architectural patterns, defines the recurring technical bottleneck of visual grounding², and, through its platform coverage, reveals the gap that motivates the present study. The work is organized here by the platform each system targets — web, mobile, and desktop — together with the vision-language foundation models on which many depend.

Web Agents

The web has been the most heavily studied environment for language-guided agents. Mind2Web, introduced by Deng and colleagues [7], is the first dataset for developing generalist web agents that follow language instructions across diverse real-world websites, comprising over two thousand open-ended tasks

collected from 137 websites across 31 domains; its accompanying two-stage model uses a small language model to filter candidate elements from large documents before invoking a larger model to choose among them. WebArena, introduced by Zhou and colleagues [8], provides a fully self-hosted, reproducible web environment of functional website clones and evaluates 812 long-horizon tasks for functional correctness rather than trajectory similarity, establishing a challenging benchmark on which contemporary models achieved completion rates far below human performance. Zheng and colleagues [9], in their SeeAct framework, decouple visual understanding from action: the model observes a screenshot, generates a textual plan, and a separate step grounds that plan onto page elements. Their central finding is diagnostic for the entire field — with oracle grounding provided, the model completes 51.1% of tasks on live websites, substantially outperforming a text-only baseline at 13.3%, but performance degrades sharply when grounding must itself be inferred, confirming visual grounding as the principal bottleneck.

Mobile Agents

Mobile-device control has produced large-scale datasets and progressively more autonomous agents. The pre-LLM work of Li and colleagues [19], discussed in §2.3, established the grounding problem for mobile interfaces. Android in the Wild, from Rawles and colleagues [10], contributes a dataset of 715,000 episodes spanning 30,000 unique instructions, in which actions are gestures at arbitrary screen coordinates rather than operations on indexed interface elements, exposing the challenge that actions must be inferred from visual appearance rather than from a structured model of the interface. AppAgent, from Zhang and colleagues [11], operates Android applications through a simplified action space of taps and swipes and learns to use new applications through autonomous exploration or by observing human demonstrations, generating application-specific documentation that matched or exceeded human-written documentation. Mobile-Agent, from Wang and colleagues [12], adopts a purely vision-centric approach that requires no system metadata, using optical-character-recognition and icon-detection modules to locate interface elements — achieving broad applicability across mobile environments at the cost of vision-grounding errors that require recovery mechanisms.

Desktop and Cross-Application Agents

Desktop agents are the most direct graphical analogue to the present work, and their limitations are the most instructive. ScreenAgent, from Niu and colleagues [14], is a vision-language-model-driven agent for cross-application desktop control that operates in a plan–act–reflect cycle, specifying actions as coordinates and key sequences; its authors acknowledge that vision-only grounding is less reliable than approaches with API access. WinClick, from Hui and colleagues [15], targets Windows-desktop grounding and observes that most existing GUI agents rely on structured data formats such as the document object model or HyperText Markup Language, which are unavailable across general desktop environments; it

therefore performs visual grounding directly from screenshots. UFO², the "Desktop AgentOS" of Zhang and colleagues [16], is the most ambitious desktop system surveyed: it fuses the native Windows accessibility framework with vision-based parsing in a hybrid action mechanism, dispatching subtasks from a coordinating agent to application-specialized agents. Its authors argue that existing computer-using agents are hindered by shallow operating-system integration, fragile screenshot-based interaction, and disruptive execution, and they identify adaptation to other desktop platforms, such as Linux and macOS, as future work. UFO² nonetheless operates as an automation layer dispatched atop the existing Windows shell rather than as a replacement for it, and its authors frame cross-platform deployment as groundwork for a unified ecosystem of desktop automation solutions — the trajectory from which the present study departs.

Contemporaneous Commercial Systems (as of September 2026)

The systems surveyed above are research artifacts. A parallel commercial development is moving faster than the peer-reviewed literature and warrants explicit positioning; what follows reflects that landscape as of September 2026 and is dated deliberately, because it will continue to move. Several products now describe themselves as agentic operating systems. WarmWind [30], released in 2026, runs autonomous workers that operate applications through vision and simulated mouse and keyboard input, hosted in a managed cloud environment built on a Linux base and aimed at automating repetitive office work. Alibaba's ANOLISA ships an AI-native Linux terminal in which shell commands and ordinary language share a single prompt, with the agent requesting approval before risky operations and a checkpoint facility providing rollback [29]. Major platform vendors have each shipped or previewed some form of computer control, ranging from screenshot-driven graphical automation to structured application-intent dispatch. Two properties separate all of them from the present work, and both are structural rather than incidental. First, each routes inference through remote infrastructure, whereas the design constraint here is that the model runs on the user's own hardware, with no network dependency. Second, each is built to act on the user's behalf, largely unattended, whereas the interface studied here is operated by a person turn by turn with every irreversible action gated on that person's approval. The distinction is not which approach is better but which question is being asked: these systems pursue autonomy, while this study asks whether conversation is a better interface for a human who is present and in control.

Vision-Language Foundation Models

Underlying many of these agents are vision-language models trained specifically for interface understanding. CogAgent, from Hong and colleagues [13], is an eighteen-billion-parameter visual language model specialized for GUI understanding and navigation, using both low- and high-resolution image encoders to recognize the small icons and text typical of graphical interfaces. It supports the position that pure-vision GUI agents are viable when the underlying model is trained appropriately — but at a compute

cost, and with the cross-platform generalization limitations common to the entire graphical-agent literature. Across web, mobile, and desktop, these systems demonstrate broad capability within a single platform class, while consistently identifying visual grounding as an unsolved bottleneck and trending toward hybrid execution that prefers structured access where available and falls back to vision.

2.6 Accessibility and Speech-Driven Interaction

A distinct thread of the literature approaches conversational computing from the perspective of accessibility, targeting users with motor, speech, or cognitive needs. This thread is relevant to the present study in two ways: it provides the most rigorous human-centered evaluations in the field, and it exhibits a limitation that is the mirror image of the mainstream agent literature, sharpening the opportunity the proposed system addresses.

VoicePilot, from Padmanabha and colleagues [5], is a framework for using LLMs as speech interfaces to physically assistive robots for individuals with motor impairments. Although its immediate target is robotic assistance rather than desktop computing, it is included here because it is the most rigorous human-centered evaluation in the LLM-speech-interface literature. The authors derive five concrete design guidelines from a study with motor-impaired participants, among them the importance of execution speed approaching that of a caregiver and the necessity of preview-and-confirm workflows — a guideline that directly informs the reversibility-based confirmation design of the present study. DexAssist, from Mehendale and Walishetti [6], targets web navigation by users with fine-motor impairments using two LLMs in tandem — one to interpret voice commands and a second to map the interpretation onto interface elements with fallback strategies — reducing the misinterpretation failures of single-model voice navigation on dense layouts. The architecture of van Dam [4] proposes retrofitting LLM-based conversational interfaces onto existing single-application graphical interfaces, exposing an application's navigation graph through a tool protocol and handling voice input through real-time speech recognition; its author observes that most production applications were never designed with speech in mind, making such retrofitting an ongoing challenge.

More recent work continues to mature the components required for speech-driven interface control. Han and colleagues, in GUIRoboTron-Speech [21], advance automated GUI agents driven directly by speech instructions, while Park and colleagues, in R-VLM [22], contribute region-aware visual grounding for more precise identification of on-screen elements. What unites this thread is that it operates at the level of the individual application or interaction technique rather than at the level of the session itself. These approaches presuppose an underlying conversational system surface on which to run, yet no such surface exists at the session level. This complementarity is significant: a general-purpose conversational interface layer would serve not only mainstream users but also provide the substrate that accessibility-

specific configurations require — positioning accessibility not as the immediate target of the present study, which scopes it out for recruitment and ethics reasons, but as a natural and well-grounded extension of its contribution.

2.7 Evaluation Environments and Benchmarks

The rapid proliferation of computer-using agents has been accompanied by increasingly rigorous evaluation environments, and these establish both the current ceiling of the field's capability and the methods by which that capability is measured. The most consequential is OSWorld, from Xie and colleagues [17], a benchmark of 369 real-world computer tasks in a real-computer environment spanning Ubuntu, Windows, and macOS. At its 2024 release the margin it measured was stark: humans achieved 72.36% success where the best evaluated model reached 12.24%. That particular margin has since closed, with leading agents now exceeding the original human baseline on the revised task set — but the closure reflects progress on short, self-contained tasks rather than on sustained computer work. Its successor, OSWorld 2.0 [26], evaluates 108 long-horizon workflows whose median item requires roughly an hour and a half of skilled human operation; there the best frontier configuration completes only 20.6% of tasks outright. The reported failures concentrate not in interface navigation but in tracking constraints across a long task, absorbing information that arrives mid-way, and verifying a result before declaring completion. The field's ceiling has moved substantially; it has not been reached. OSWorld is also directly relevant to the present study as the source of a real-task suite against which benchmark comparability can be established.

Complementing the benchmark literature, the survey of LLM-brained GUI agents by Zhang and colleagues [18] synthesizes the field through 2024, organizing it around the components of perception, planning, action, and memory, and providing the four-layer vocabulary adopted in §2.2 of this review. It enumerates the field's open challenges — accurate element localization, knowledge retrieval, long-horizon planning, and safety-aware execution control — Notably, none of the challenges it enumerates concerns replacing the session layer itself: the survey takes the surrounding operating-system session as given, which is the assumption the present study sets out to test. What the evaluation literature reveals, however, is a specific methodological limitation: rigorous as these benchmarks are, they measure the autonomous task completion of agents acting on a user's behalf, not the comparative performance of a human user working through a conversational interface versus a conventional graphical one. That distinction motivates the evaluation design reviewed in §2.9.

2.8 Synthesis: Comparative Analysis and Research Gap

Positioning the surveyed works against one another along the four-layer pipeline makes the structure of the field, and the gap within it, explicit. Table 2.1 summarizes the principal systems by coverage scope, action mechanism, evaluation approach, and key limitation.

Table 2.1. Comparative analysis of surveyed systems on conversational and LLM-mediated interfaces for personal computing, positioned by coverage scope, action mechanism, evaluation, and key limitation.

Ref.	System	Year	Coverage scope	Action mechanism	Evaluation	Key limitation
[1]	Unix Consultant	1988	Shell consultation	Natural-language Q&A (no execution)	Demonstration, case studies	Hand-engineered knowledge base
[2]	Westenfelder et al.	2025	Shell (CLI)	Command translation	600-pair test set, fine-tuning	Limited system-context awareness
[3]	NaSh	2025	Shell (Linux)	Guarded command translation	Worked examples, design paper	No formal user evaluation
[4]	Multimodal GUI Arch.	2025	Single application	API binding + voice	Architectural prototype	Per-application retrofitting
[5]	VoicePilot	2024	Assistive robot	Voice + robot API	User study, motor-impaired participants	Robot domain, not the OS
[6]	DexAssist	2024	Web (e-commerce)	Dual-LLM voice + element mapping	Web-task evaluation	Narrow to e-commerce
[7]	Mind2Web	2023	Web (137 sites)	Element selection from filtered HTML	Dataset benchmark	Structured-data-only in original form
[8]	WebArena	2024	Web (self-hosted)	Functional task execution	Benchmark, low model baseline	Simulated, not live web
[9]	SeeAct	2024	Web (live sites)	Visual plan + grounding	51.1% with oracle grounding	Grounding hard without oracle
[10]	Android in the Wild	2023	Mobile (Android)	Coordinate gestures	Dataset, 715k episodes	Brittle to interface updates
[11]	AppAgent	2023	Mobile (10 apps)	Tap/swipe + element labels	50-task evaluation	Per-app knowledge acquisition

Ref.	System	Year	Coverage scope	Action mechanism	Evaluation	Key limitation
[12]	Mobile-Agent	2024	Mobile (vision-only)	OCR + icon detection + planning	Mobile-Eval benchmark	Vision-grounding errors
[13]	CogAgent	2024	Cross-platform GUI	Vision-language model (high-res)	SOTA on VQA / navigation	Compute cost of high-res inputs
[14]	ScreenAgent	2024	Cross-app desktop	Vision + mouse/keyboard	Plan-act-reflect evaluation	Vision-only grounding fragility
[15]	WinClick	2025	Desktop (Windows)	Visual grounding	WinSpot benchmark	Windows-specific
[16]	UFO ²	2025	Desktop (Windows)	Hybrid API + vision	System deployment, task benchmarks	Windows-only; automates atop the OS
[17]	OSWorld	2024	Ubuntu, Windows, macOS	Evaluation harness	369 tasks; 12.24% best vs 72.36% human at 2024 release	Reveals field-wide grounding gap
[19]	PixelHelp / seq2act	2020	Mobile (Pixel)	Two-stage transformer	ACL benchmark, three datasets	Pre-LLM; dataset-bound
[20]	ReAct	2023	Cross-domain	Thought-action-observation prompting	Multi-benchmark evaluation	Inherits LLM hallucination, brittleness
[21]	GUIRoboTron-Speech	2025	GUI (speech-driven)	Speech-instructed agent	Benchmark evaluation	Application, not session, level
[22]	R-VLM	2025	GUI grounding	Region-aware visual grounding	Grounding benchmark	Component, not full system
[24]	Local LLMs for NL2Bash	2026	Shell (CLI)	Open-source model evaluation	Large-scale model comparison	Evaluation, not a deployed system

Six patterns emerge from this comparison, each surfacing a recurring theme in the literature.

Bifurcation by Coverage Scope

The field is bifurcated by coverage scope. Shell-level work [2, 3] and single-application work [4, 5] sit at the narrow end; cross-application desktop, web, and mobile agents sit at the broad end but each within a single platform class. No surveyed system implements a working conversational replacement for the full graphical session that has been empirically validated against the conventional desktop workflow it

would displace. The Unix Consultant [1] came closest in ambition four decades ago, but its pre-LLM foundation prevented it from scaling beyond demonstration. The absence is structural rather than incidental. Even UFO² [16], the most operationally advanced desktop-agent system surveyed, is by construction an automation layer dispatched atop the existing Windows shell, and its stated future direction is broader cross-platform automation rather than replacement of the session itself. The substitution this study attempts is therefore unclaimed, not merely under-published.

Execution Mechanism at the Action Layer

A second pattern concerns the execution mechanism at the Action layer. The literature divides among command translation, API binding, and GUI simulation, with a dominant trend toward hybrid execution [16, 4] that prefers structured access where available and falls back to vision. Vision-only systems [12, 14] achieve broad applicability at the cost of grounding fragility, while structured-access systems achieve precision at the cost of per-environment integration. Notably, the Unix Consultant operated purely at the consultation layer without executing actions — an approach that reflects an underappreciated design principle, that the agent that explains can be safer than the agent that acts, which the confirmation-gated design of the present study revisits.

Visual Grounding as the Principal Bottleneck

A third pattern is that visual grounding is the field's principal bottleneck, identified explicitly by both SeeAct [9] and the GUI-agents survey [18]. The 51.1% ceiling of GPT-4V with oracle grounding and its sharp degradation without are attributable in significant part to grounding failure. This bottleneck is a property of the graphical-simulation approach specifically; a command-line approach that grounds intent into shell commands rather than into on-screen pixel regions sidesteps it, at the cost of being bounded by what is expressible on the command line. Direct comparison qualifies that tradeoff usefully. Across 440 matched desktop tasks, screen-only graphical agents pass 59.1% while command-line agents reach only 48.2% — the shell is not automatically the stronger substrate — but command-line agents augmented with result verification reach 69.3%, the highest of the three conditions [27]. Execution through the shell becomes advantageous when it is paired with a mechanism that checks what the command actually did, which is precisely the role a reversibility gate and an undo record play in the present design.

Scarcity of Comparative User Evaluation

A fourth pattern concerns evaluation method. Most surveyed systems are evaluated either through architectural demonstration [3, 4] or through curated task benchmarks [7, 8, 12, 17]. User studies are rare, and the comparisons that do exist are narrow. VoicePilot [5] conducts a substantive human-centered

evaluation of LLM-based interaction, and AXIS [25] compares an API-first agent against unaided manual operation in a within-subjects design — but within a single application, on a single platform, with a cloud-hosted frontier model. No surveyed work compares a conversational interface against the conventional graphical workflow across operating-system-level tasks on the same population. This is the empirical gap best filled by a controlled study of conversational interfaces.

The Operating-System Substrate

A fifth pattern concerns the operating-system substrate. Windows [15, 16], Android [10, 11, 12], and the web [7, 8, 9] are each heavily represented; the Linux work that exists is aimed elsewhere than the end user's desktop session. NaSh [3] addresses Linux at the shell level as a design proposal; ANOLISA [29] ships a natural-language Linux terminal with approval gating and rollback, but scoped to server and DevOps workloads; and OSWorld [17] includes Ubuntu as an evaluation platform rather than a deployment target. No surveyed system implements a conversational session layer for the Linux desktop addressed to the person using the machine, despite Linux serving as the substrate for most developer and server-side workflows.

Shared Lineage and Its Fragility

A sixth pattern is the field's shared intellectual lineage and its attendant fragility. As established in §2.3, nearly every post-2023 system inherits the ReAct [20] convention of interleaving reasoning with action, which explains both the field's rapid progress and the pervasiveness of LLM-reasoning failure modes across it — a fragility that makes safety and recoverability mechanisms, rather than raw capability alone, decisive for any deployable system.

A related but distinct class of contemporary systems warrants explicit positioning here: AI-assisted coding agents such as Claude Code, Cursor, and Aider, which translate natural-language intent into shell commands and execute them under human oversight — the same basic mechanism the present study adopts. These tools are excluded from Table 2.1 not on the grounds of being industry rather than academic work — a substantial share of the literature surveyed here is likewise unrefereed preprint — but because they occupy a different position on every axis that matters to this study. They are built for software engineers operating within an existing code repository, are powered by large models hosted on remote infrastructure, and run as a tool installed atop an otherwise unmodified desktop session. SynapseOS targets the inverse on each axis: any user of the machine regardless of command-line fluency, a locally-hosted small language model rather than a cloud dependency, and replacement of the session layer itself rather than an application running within it. The mechanism is shared; the population, the model's location, and the system's place in the stack are not.

These patterns converge on a threefold research gap. *First*, no system in the literature provides a conversational interface as a general route to operating a personal computer — one addressed to the person using the machine, spanning the operating system rather than a single application, and evaluated as an interface rather than as an autonomous agent. Existing systems are automation utilities: they act on the user’s behalf, and they are measured by how often they complete a task unattended rather than by whether a person directs them more effectively than they direct the interfaces they already have. *Second*, the work that exists on an open platform targets servers, agents, and benchmarks rather than the desktop session of an end user — and the platform constraint runs deeper than coverage. An interface layer that observes and acts on a desktop session can only be built where the platform permits it; Windows and macOS are proprietary and do not. Linux is therefore the condition of the experiment rather than its subject, and the absence of end-user desktop work there is what this study addresses. *Third*, controlled comparison against the conventional graphical workflow exists only within a single application, on a single platform, with a cloud-hosted frontier model [25]; no such comparison exists for operating-system-level tasks under a locally-hosted model and an explicit safety gate. SynapseOS — a conversational interface layer for the Linux desktop that translates natural-language intent into shell commands under a reversibility-based confirmation gate, evaluated against each participant’s own operating system — is positioned precisely against these three gaps. Its relationship to the closest prior work is worth stating directly. NaSh [3] proposes guardrails for an LLM-powered natural language shell — generated code that remains analyzable, selective undo, summaries of what a command would affect — as a design position, and explicitly leaves implementation and empirical evaluation as open work. The present study is an implementation of that position, substituting a concrete reversibility taxonomy and a four-mechanism undo journal for the general principle, and supplying the user evaluation NaSh defers.

2.9 Review of the Methodological Literature

Beyond the systems it proposes to build upon, the present study draws on a methodological literature that shapes how a conversational interface should be evaluated. The evaluation practices of the surveyed work fall into three categories, each with a characteristic limitation. Architectural demonstrations [3, 4] establish feasibility but provide no performance evidence. Curated task benchmarks [7, 8, 17] provide reproducible, quantitative measures but assess the autonomous task completion of agents acting on a user’s behalf rather than the performance of a human working through an interface. Human-centered user studies, exemplified by VoicePilot [5] and AXIS [25], provide the most direct evidence of an interface’s value to real users but remain rare in this literature and, where present, are confined to a single application rather than the operating system as a whole. The instruments this study adopts are drawn from established

practice rather than constructed for it: the System Usability Scale [40] for perceived usability, with its interpretation grounded in subsequent large-sample analysis [42, 43], and the NASA Task Load Index [41] for perceived workload.

This critique motivates the methodological posture of the present study, which is grounded in established human-computer-interaction evaluation practice. The study adopts a *within-subjects* design, in which each participant completes tasks under both the SynapseOS condition and the condition of their own operating system, so that each participant serves as their own control and between-person variance is removed as a confound; the order of conditions is counterbalanced to distribute learning effects evenly. It further adopts an *expert-baseline* comparison — using each participant's most familiar environment as the comparison condition rather than a fixed alternative that may be unfamiliar — so that observed differences reflect interface quality rather than differential familiarity with the operating system, strengthening the ecological validity that the benchmark-only evaluations in the surveyed literature lack. To integrate the quantitative measures of task completion time and error rate with qualitative evidence of user experience, the study employs a *convergent mixed-methods* structure in the sense set out by Creswell and Plano Clark [44], collecting both strands concurrently and integrating them at the interpretation stage; the qualitative data, gathered through retrospective think-aloud over the screen recording and a semi-structured interview, are analyzed using the thematic-analysis method of Braun and Clarke [23], a widely adopted six-phase approach for identifying and reporting patterns in qualitative data. Design considerations surfaced by the reviewed systems inform the evaluation directly: the preview-and-confirm guideline of VoicePilot [5] and the reversibility-and-guardrail framing of NaSh [3] together motivate an explicit validation of the confirmation gate and undo mechanism as part of the study.

The specific instantiation of this methodology — the composition of the benchmark and custom task suite, the participant profile and screening criteria, the session procedure, the metrics and their computation, and the statistical and qualitative analysis plan — is presented in detail in Chapter 3.

2.10 Summary

This chapter reviewed the literature on natural-language and LLM-mediated interfaces to personal computing across nearly four decades, organized along a four-layer pipeline of Input, Reasoning, Action, and Operating-System Integration. The historical foundations established the long-standing aspiration to a natural-language interface to a real operating system [1] and the methodological shift, driven by in-context reasoning [20], that made the current generation of systems possible. The shell-and-command-line thread most directly related to the proposed design demonstrated that natural-language-to-Bash translation is tractable [2], that safety and reversibility are central concerns [3], and that local deployment is feasible [24], while leaving the extension from shell to session, and empirical user evaluation, as open work. The graphical-agent literature across web, mobile, and desktop [7–16] established the field's architectural

patterns and its recurring bottleneck of visual grounding, while the accessibility thread [4, 5, 6, 21, 22] revealed a complementary limitation that a general conversational interface layer would resolve. The evaluation literature [17, 18, 26] traced a capability ceiling that has risen sharply on short tasks while remaining low on long-horizon work, and exposed a methodological gap: interfaces are seldom evaluated comparatively against the conventional workflow with real users, and never yet at the level of the operating system itself.

Synthesized, the surveyed works reveal a threefold gap: no conversational replacement for the graphical session layer, no coverage of the Linux desktop, and no controlled comparison of a conversational interface against the conventional workflow across user populations. The present study addresses all three through SynapseOS, a conversational interface layer for the Linux desktop evaluated by controlled, within-subjects comparison against each participant's own operating system. The methodology through which this evaluation is conducted is the subject of Chapter 3.

Notes

1. A computer-using agent is an artificial-intelligence system that operates a computing environment by issuing actions and observing their results. A GUI agent is a computer-using agent whose action space consists of operations on a graphical user interface — typically clicks, key presses, scrolls, and text entry — usually driven by a large language model or a multimodal (vision-language) model that additionally accepts images as input.
2. Grounding is the problem of mapping a model's textual plan onto a concrete operation in the environment. Visual grounding is the specific subproblem of identifying the on-screen pixel region corresponding to a referenced interface element — repeatedly identified in the surveyed literature as the principal bottleneck for graphical agents, and one that a command-line approach avoids by grounding intent into shell commands rather than into screen coordinates.

Chapter 3

METHODOLOGY

This chapter describes the research methodology employed in designing, implementing, and evaluating SynapseOS — an agentic conversational interface layer that runs over an ordinary Linux desktop session rather than replacing it. The methodology is organized into three sections: Dataset, which defines the data to be collected and how it will be gathered; Procedure, which describes the step-by-step process of building the system and conducting the study; and System Testing, which defines the evaluation design, metrics, and statistical approach used to compare SynapseOS against each participant's own primary operating system.

This methodology answers the four research questions posed in Chapter 1. *RQ1* asks whether the recoverability of an arbitrary generated command can be determined automatically, and a minimal recovery plan computed for it, more accurately than by matching against an enumerated list of destructive patterns. *RQ2* asks how a conversational interface layer can be designed so that a person can direct an operating system in ordinary language without knowing the syntax its command line requires; *RQ3*, what confirmation and recovery design allows a user to act on their own system through natural language while preserving the ability to undo what was done, including when a command was approved in error; and *RQ4*, whether a conversational interface narrows the gap between what technically fluent users and users fluent in neither existing interface can accomplish, and how the resulting experience compares to the interface each participant uses today.

The study therefore makes two contributions of different kinds, established by two independent evaluations, and they are reported separately for that reason. Section 3.1 defines the corpus-based evaluation that answers *RQ1* without reference to any participant; Section 1 defines the data gathered to answer *RQ3* and *RQ4*; Section 2 describes the system built to answer *RQ2* and *RQ3*; and Sections 3.2 onward define the user study that answers *RQ3* and *RQ4*. Neither evaluation depends on the other: the algorithm takes a shell command as a string and is indifferent to what produced it, so its corpus, baselines, and results stand independently of the interface in which it is deployed, while the comparative study addresses a gap in the literature — conversational interfaces evaluated across user populations of differing technical fluency — that the algorithm does not speak to at all.

The two evaluations are independent by construction, and this is the reason the questions are ordered as they are. Section 3.1 defines the corpus-based evaluation that answers RQ1 without reference to any participant; Section 1 defines the data gathered to answer RQ3 and RQ4; Section 2 describes the system built to answer RQ2 and RQ3; and Sections 3.2 onward define the user study that answers RQ3 and RQ4. Removing the conversational interface from this study would remove RQ2 through RQ4 and the user study with them; RQ1, its corpus, its baselines, and its results would remain intact, because the analysis takes a shell command as a string and is indifferent to what produced it.

Conceptual Framework

SynapseOS operates as a conversational shell layer positioned between the user and the Linux command-line environment. The user issues natural language commands through a persistent chat interface. An intent parsing pipeline — powered by a locally-hosted small language model — translates each utterance into a shell command, which is executed in a bash subprocess. Output is captured and rendered back in the conversational interface. The user issues intent in natural language and receives results in natural language. The generated shell command is nonetheless displayed rather than hidden: the confirmation gate requires the user to approve irreversible operations, and an approval given without sight of what is being approved is not consent. Visibility also makes the system's translation of intent observable to the participant, which is what allows a user to notice a misinterpretation before it executes rather than after. Between translation and execution sits the recoverability analyser, which is where this study's algorithmic contribution operates: before a command is dispatched, its effects are derived from its syntactic structure, a verdict is computed over them, and where the command is reversible only with preparation, the pre-images that make it so are captured first. The confirmation gate consumes that verdict rather than a pattern match. The analyser is specified in Section 2.1(c) and evaluated in Section 3.1, independently of the user study. This architecture is illustrated in Figure 3.1.

The framework is evaluated through a controlled user study in which participants complete a fixed set of tasks under two conditions: using SynapseOS as the interface, and using their own primary operating system — Windows, macOS, or Linux — as the baseline. This expert baseline design pits SynapseOS against the environment each participant already knows and uses daily, generating empirical data on task completion time, error rate, and user satisfaction across the real-world OS landscape [17].

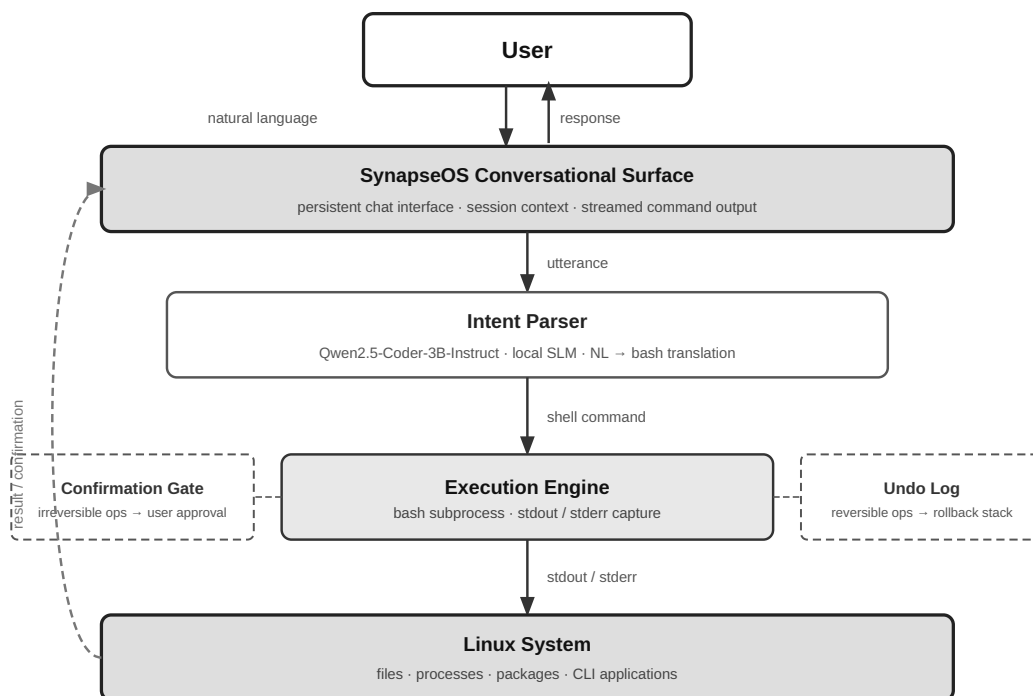


Figure 3.1. Conceptual framework of SynapseOS. The user issues a natural language command to the conversational surface, which routes the utterance to the intent parser (Qwen2.5-Coder-3B-Instruct, local SLM). The intent parser produces a shell command dispatched to the execution engine (bash subprocess). Irreversible operations are intercepted by the confirmation gate; reversible operations are logged to the undo stack. Command output (stdout/stderr) is captured, rendered in the conversational surface, and presented back to the user — completing the interaction loop.

System Environment Specifications

The user study is conducted across three dedicated machines — one running SynapseOS (Condition A) and two covering the three supported baseline operating systems (Condition B). Table 3.1 describes each machine's role and operating environment. Exact processor, RAM, and storage specifications for each machine are confirmed and documented in the study protocol prior to the pilot study. Table 3.2 specifies the minimum hardware and software requirements for deploying SynapseOS independently of the study configuration.

Table 3.1. Study evaluation machines

Machine	Role	OS	Notes
SynapseOS evaluation machine	Condition A — all participants	SynapseOS	GUI mode; fullscreen conversational interface running within an XFCE (X11 session) desktop environment; based on Debian 13 (Trixie); Ollama + Qwen2.5-Coder-3B-Instruct running locally; a participant-accessible fallback returns

			to the underlying XFCE session if needed, logged and excluded from the primary analysis (Section 3.6)
Windows / Linux dual-boot baseline machine	Condition B — Windows- or Linux-primary participants	Windows 11 or Ubuntu 24.04 LTS (GNOME), selected at boot	Standard consumer x86-64 hardware; Windows 11 and Ubuntu installed on separate partitions; standard user account per OS; no custom software. Facilitator boots to the OS matching the participant's assignment before the session begins.
macOS baseline machine	Condition B — macOS-primary participants	macOS (current release)	Apple Silicon hardware; standard user account

Table 3.2. SynapseOS minimum deployment requirements. SynapseOS is distributed as a Linux system based on Debian 13 (Trixie).

Component	Minimum Requirement
Processor	64-bit x86 processor, 2-core minimum; no GPU required
RAM	4 GB (CLI mode ⁸ / TUI mode ¹); 8 GB recommended (GUI mode)
Storage	10 GB available (operating system, language model, and application)
Display	CLI mode / TUI mode: any terminal or remote connection; no graphical environment required. GUI mode: graphical desktop environment required.
Language Model	Locally-hosted small language model (~1.8 GB) ² ; CPU-only inference; no GPU required. A cloud language model backend is planned as an optional, user-configured alternative; it is not implemented in the thesis prototype.
Shell	GNU bash. Commands generated by the intent parser are dispatched through <code>bash -c</code> ; a POSIX-only shell such as dash is insufficient, since generated commands may use bash-specific syntax.
Sandbox	bubblewrap. The analysis runs its read-only resolvers with the whole filesystem mounted read-only inside it. Without it, resolution is switched off and any command that needs it asks, so the system stays safe and only becomes more cautious.
Inference Server	Local inference server (Ollama) ³ ; manages model lifecycle, loading, and query serving on the host machine.

The user study evaluates SynapseOS in GUI mode, which requires a graphical desktop environment and 8 GB RAM. All three interface modes are deliverables of this study; the study condition, however, depends on GUI mode being complete and stable in advance of participant recruitment. Because a slipped implementation milestone would otherwise delay the entire evaluation, the study condition is fixed at a defined readiness checkpoint prior to ethics submission: if GUI mode is not feature-complete and stable at that checkpoint, the study is conducted in TUI mode instead, which is functionally equivalent in every respect the research questions address — the same conversational surface, the same execution pipeline, the same confirmation gate and undo journal, differing only in the rendering layer and in requiring no display server. The mode actually used is reported with the results. If SynapseOS becomes unresponsive during a task or a participant wishes to stop using it, a fallback returns them to the underlying XFCE

session, which continues running beneath the conversational interface throughout Condition A; every fallback invocation is logged, and any task during which it occurs is excluded from the primary completion-time and error-rate comparison rather than counted as a SynapseOS success (Section 3.6). SynapseOS additionally supports a TUI mode — a terminal-based interface requiring no display server, running on as little as 2 vCPU and 4 GB RAM — designed for server and remote deployment environments accessible over SSH, and a CLI mode⁸ — a single non-interactive invocation, with no session persisting between separate calls, that runs a bounded loop toward one natural-language request: propose a command, pass it through the same reversibility-based confirmation gate described in the Execution Pipeline (Section 2.1) — executing immediately if reversible, blocking pending explicit confirmation if not — then feed the result back so the model can propose the next step or signal completion, up to a fixed step limit, before the invocation exits, sharing TUI mode's minimal hardware footprint but intended for scripting and automation rather than an interactive session. Neither CLI nor TUI mode is evaluated in the user study.

Section 1: Dataset

The data gathered for this study falls into three categories: benchmark task data used to evaluate the system in a standardized environment, user study data collected from human participants in a controlled setting, and system performance data captured automatically through built-in telemetry during all sessions.

1.1 What to Gather

a) Benchmark Task Suite

The primary benchmark dataset is the OSWorld task suite [17], which provides standardized real-computer tasks across shell operations, file management, web browsing, and application-level interactions. OSWorld is the only existing benchmark that evaluates computer-using agents in real operating system environments, making it the most appropriate external standard for SynapseOS evaluation. Tasks are presented as natural language instructions; agents interact with a real virtual machine environment, and outcomes are evaluated by automated inspection of the resulting system state — whether the goal was actually achieved — rather than by comparing individual actions. In addition to OSWorld tasks, a custom cross-platform task suite is developed for this study, covering file search and organization, system and process monitoring, application and package management, and text and data processing — each task defined as a human-level goal achievable via SynapseOS natural language commands and via native OS tools on the comparison machine. Each task in both suites is documented with a description, expected outcome, task category, and difficulty level.

b) User Study Data

The following data types are collected from each participant during the user study:

- **Task completion time:** the elapsed time from when the task prompt is presented to when the participant signals task completion or the session logs a terminal event
- **Error rate:** the proportion of task attempts ending in failure or an incorrect outcome, per participant per condition; recovery attempts are recorded separately as an error category rather than counted in the rate
- **User satisfaction scores:** responses to the System Usability Scale (SUS) [40] and the NASA Task Load Index (NASA-TLX) [41], collected immediately after each condition; both are validated instruments for measuring perceived usability and cognitive workload respectively
- **Qualitative feedback:** responses gathered through a semi-structured post-study interview covering perceived ease of use, trust in the system, and comparison between the two conditions

c) System Performance Data

The following telemetry data is captured automatically by SynapseOS throughout all sessions:

- Command execution latency: time elapsed from utterance submission to bash subprocess completion
- Undo log events: count and type of operations logged to the undo stack
- Confirmation prompts triggered: count of irreversible operations that required user confirmation
- Conversational turns per task: the number of utterances required to complete each task
- Intent parsing accuracy: whether the system correctly identified the user's intent on the first attempt

1.2 How to Gather

a) Benchmark Tasks

SynapseOS is deployed on the Debian 13 (Trixie) environment matching the OSWorld evaluation setup [17]. Each benchmark task is executed through the conversational interface, and results are captured programmatically through the built-in telemetry system. Each task is supplied to SynapseOS as a natural language instruction, and the resulting system state is compared against the expected outcome defined in the benchmark. Because SynapseOS gates every command classified irreversible on explicit confirmation, benchmark runs are facilitator-attended rather than unattended; the confirmation gate remains active throughout and every confirmation is logged as part of the run record. OSWorld scores the resulting system

state rather than a human operator's performance, so it applies only to the SynapseOS condition. Condition B, which a human participant operates directly, has no equivalent and is instead evaluated through the custom cross-platform task suite administered in the user study (Section 1.2b).

b) User Study

The user study is conducted in a controlled laboratory setting with two machines: the SynapseOS evaluation machine (Debian 13, conversational interface) and the baseline machine running the participant's native OS — Windows 11, macOS, or Linux with a GNOME desktop — as determined during screening. Each participant completes a matched task set on both machines under a within-subjects design. Because completing the identical task twice would measure recall rather than interface efficiency, two parallel task forms — Set A and Set B — are constructed, matched item for item on category, difficulty level, and number of required steps, differing only in the specific targets involved. Form and condition order are crossed in a 2×2 counterbalance, yielding four participant groups of ten: Set A first on SynapseOS, Set B first on SynapseOS, Set A first on the native OS, and Set B first on the native OS. Task order within a form is randomized per participant. Form equivalence is verified during the pilot study, and condition order is retained as a between-subjects factor in the analysis so that residual asymmetric transfer — carryover that is larger in one order than the other, which counterbalancing redistributes but does not remove — is detectable rather than assumed absent. Screen capture software records activity on both machines; the SynapseOS session logger supplements the recording for the SynapseOS condition.

The study involves 40 participants divided into two groups:

- **Novice users (n = 20):** participants with limited computing experience, capable only of basic tasks (web browsing, document editing) on any OS, and no prior command-line familiarity, recruited from the general university population
- **Power users (n = 20):** participants with demonstrated proficiency in computing on their primary OS — capable of performing file management, system monitoring, and process administration tasks efficiently — recruited from the university's information technology and computer science programs

Participants are recruited through university channels. Recruitment enforces a minimum quota of four participants per primary OS background (Windows, macOS, Linux), scaled with the sample so that the realized set cannot end up concentrated in a single OS and leave the per-OS exploratory subgroup analysis (Section 3.3) without data for the others. The binding constraint on this quota is the Linux-primary novice cell: a daily Linux user with no command-line familiarity is uncommon, and if that cell cannot be filled during recruitment the Linux quota is met from power users alone, with the resulting imbalance reported rather than concealed. Informed consent is obtained from all participants prior to the session. Ethics

approval is secured before recruitment begins. Participant data is pseudonymized at the point of collection under the participant ID scheme described in Section 2.5; recordings, which cannot be anonymized at collection, are governed by the retention and destruction rule stated there.

After completing each condition, participants fill out the SUS and NASA-TLX questionnaires. After completing both conditions, a brief semi-structured interview is conducted to gather qualitative feedback.

1.3 Why

The OSWorld benchmark is used as the primary evaluation standard because it is the only existing benchmark that evaluates computer-using agents in real operating system environments with real applications [17]. Its use allows SynapseOS results to be situated within the existing literature.

A custom cross-platform task suite is necessary because OSWorld's task coverage does not represent the human-goal-framed system tasks relevant to this study — file organization, system monitoring, process management, and text processing — achievable both through SynapseOS natural language commands and through participants' native OS tools (Windows, macOS, or Linux graphical desktop). Tasks are scoped to operations achievable via bash on SynapseOS and via native graphical or terminal tools on the comparison machine.

The evaluation follows a convergent (parallel) mixed-methods design: the quantitative performance measures — task completion time, error rate, SUS, and NASA-TLX — and the qualitative interview data are collected concurrently within each session and integrated at the interpretation stage, so that the measured differences between conditions and participants' accounts of why those differences arise inform one another rather than being read in isolation.

The within-subjects design is chosen because it allows each participant to serve as their own control, producing a direct comparison between conditions while requiring a smaller participant pool than a between-subjects design. Counterbalancing controls for the primary threat to internal validity: the learning effect, wherein participants may perform better in the second condition simply due to familiarity with the tasks.

The expert baseline design — using each participant's primary OS as Condition B rather than a fixed Linux graphical desktop — is chosen because a fixed Linux baseline would confound interface quality with OS familiarity. The overwhelming majority of participants have never used a Linux desktop; performance under that condition would reflect unfamiliarity with a foreign environment, not a meaningful comparison of interface paradigms. The stronger and more ecologically valid test is whether SynapseOS can compete with what participants already know and use daily. A positive result against a participant's native environment is a more compelling argument for SynapseOS than a win against a system participants are encountering for the first time.

The dual-population design — novice and power users — directly addresses the gap identified in Chapter 1, where no prior work has evaluated a conversational interface across user populations with differing technical backgrounds [17]. The three data categories align with the three interface questions: *RQ2* is addressed by the system design and benchmark task coverage; *RQ3* is addressed by system performance telemetry (command execution latency, intent parsing accuracy, undo and confirmation events); and *RQ4* is addressed by the user study metrics (task completion time, error rate, satisfaction scores). *RQ1* draws on none of these, being answered against the labelled command corpus described in Section 3.1.

1.4 Participant Screening Criteria

Participants are assigned to the novice or power user group based on a pre-study screening questionnaire administered before the session begins. The questionnaire assesses the following dimensions:

- Years of general computer use
- Primary OS — the operating system used as the primary daily computing environment (Windows / macOS / Linux); must average four or more hours of use per day to qualify
- Self-reported proficiency on primary OS (basic / intermediate / advanced)
- Self-reported command-line familiarity (none / basic / intermediate / advanced)
- Prior exposure to conversational AI interfaces — frequency of use (never / occasionally / regularly / daily) and which tools (chatbots, voice assistants, code-completion or agentic coding assistants, other LLM-based tools)

Self-reported proficiency is confirmed with a short behavioral screener: three timed mini-tasks performed unaided on the participant's primary OS using its native tools, one per system-task category used in the main study — file organization (locate and relocate a set of recently modified files), process monitoring (identify the application consuming the most memory via the OS's native system monitor), and application management (determine whether a named application is installed and identify its version). Each task is scored pass/fail by the facilitator against a fixed time limit and completion criterion. The behavioral screener exists because self-reported proficiency alone is not a reliable classifier — a participant's technical background in one domain (e.g., software development) does not guarantee fluency with a given OS's native file management, process monitoring, and system administration tools, which is the specific competency this study's novice/power-user split is intended to capture.

Participants are classified as **novice users** if they report basic proficiency on their primary OS, computing use limited to web browsing and document editing, no prior command-line experience on any platform, and fail the behavioral screener. Participants are classified as **power users** if they report advanced proficiency on their primary OS — capable of performing file management, process monitoring, and system administration tasks efficiently using that OS's native tools — regardless of which OS that is, and

pass all three behavioral screener tasks. Participants whose self-report and behavioral screener results disagree, or who do not clearly satisfy either classification, are excluded. Primary OS is recorded for all participants as a grouping variable, and prior conversational AI exposure as a covariate, in the analysis; prior AI exposure is analyzed as an exploratory covariate only (see Section 3.3) rather than a primary grouping variable, since splitting the sample along a third axis would leave subgroups too small to support a hypothesis test.

1.5 Task Design

The custom cross-platform task suite is designed to cover human-level system tasks achievable both through SynapseOS natural language commands and through native tools on the participant's primary OS. Task design follows four principles:

1. **Ecological validity** — tasks reflect actions a real user would perform in their everyday computing environment: finding and organizing files, monitoring system resources, managing processes, and handling text data
2. **Cross-platform equivalence** — all tasks are defined as human-level goals achievable both through SynapseOS natural language commands (mapped to bash) and through native tools on the participant's primary OS (File Explorer, Task Manager, Finder, Activity Monitor, GNOME, or equivalent). The method of achievement differs by platform; the human goal does not
3. **Difficulty stratification** — tasks are rated at three difficulty levels (simple, moderate, complex) based on the number of steps required and the specificity of the target operation
4. **Population appropriateness** — both participant groups receive the same task set, with task wording written in plain language accessible to novice users and free of OS-specific or CLI-specific terminology

The custom task suite comprises tasks organized into four categories: file search and organization, system and process monitoring, application and package management, and text and data processing. All tasks are defined at the human-goal level — achievable via SynapseOS natural language on the evaluation machine and via native graphical or terminal tools on the comparison machine. Each task is documented with a unique identifier, a natural language prompt as presented to the participant, the expected outcome, and the assigned difficulty level. The full suite — twelve core tasks in two parallel forms, plus four boundary tasks lying outside shell expressibility that are scored and reported separately — is given in Appendix C. Difficulty calibration and form equivalence are validated during the pilot study and revised as necessary before the main study begins.

Section 2: Procedure / Experiment

Building SynapseOS and running the study that evaluates it are two related but distinct efforts. The system itself — the conversational interface layer and execution pipeline — is designed and implemented first, followed by the scaffolding that makes the resulting study trustworthy: collected data is processed and pseudonymized, the system and instruments are validated, each participant session follows a standardized procedure, and participant welfare is protected throughout.

2.1 System Design and Implementation

The construction of SynapseOS proceeds across two primary workstreams — the conversational interface layer and the execution pipeline — integrated into a unified session manager that will launch in place of the conventional Linux graphical desktop.

a) Conversational Interface Layer

The conversational surface is designed as a persistent, full-screen chat interface that serves as the primary and sole user-facing interface when SynapseOS is active as the session manager. The CLI and TUI interface modes implementing this surface are complete; the full-screen GUI mode is in development, and the readiness checkpoint governing its use in the study is described in the System Environment Specifications. The interface accepts natural language input, displays system responses and command output, and maintains conversational context across the session. An intent parsing pipeline translates each user utterance into a shell command for execution.

Conversational context is session-scoped: history is held in memory for the duration of the login session and is cleared on logout or reboot, with no persistence layer in the thesis prototype. When accumulated history exceeds a fixed token budget — set well below the 8,192-token context window configured for the intent parsing model, which itself supports up to 32,768 tokens — a rolling window discards the oldest turns to keep the conversation within budget. Verbose command output — such as a full directory listing or search result — is truncated to a compact summary before being appended to history, preventing a single command's output from consuming a disproportionate share of the available context.

SynapseOS is designed around a locally-hosted small language model (SLM) as the default, model-agnostic inference backend, with cloud inference available only as an explicit opt-in. Because the system operates at the OS level — processing every natural language command and observing terminal output, file paths, command history, and interaction patterns — routing inference through a cloud API would give an external provider continuous visibility into the user's computing activity. On-device inference eliminates this exposure by default; the architecture reserves the option for a user to supply an API key for a supported cloud provider and substitute a hosted frontier model, entirely at their discretion;

this path is not implemented in the thesis prototype. The default configuration requires no API key, no internet connection, and no per-query cost, making SynapseOS fully operational in offline and air-gapped environments.

The primary SLM is Qwen2.5-Coder-3B-Instruct, an open-weights⁴ code-specialized model that demonstrated the strongest natural language-to-bash translation accuracy among models below 7B parameters in controlled evaluation [2]. Among the models directly compared in that evaluation, its code-specialized pretraining produced a clear margin over general-purpose models of comparable or larger size — 58% NL2Bash accuracy with prompt engineering, versus 49% for Llama 3.2-3B and 37% for a LoRA-fine-tuned Llama 3.2-1B — attributable to Qwen2.5-Coder's training corpus of 5.5 trillion code tokens against Llama 3.2's general-purpose pretraining. At 4-bit quantization⁵, the model requires approximately 1.8 GB of memory and runs on the target Debian 13 (Trixie) hardware without a dedicated GPU. The model will be fine-tuned via Low-Rank Adaptation (LoRA)⁶ on the NL2Bash corpus⁷ and a custom SynapseOS task suite to maximize intent parsing accuracy.

Accuracy is measured using an execution-based scoring methodology rather than syntax-level string matching, following Westenfelder et al. [2] — see Section 2.3 for the full evaluation procedure.

This selection was reconfirmed against models released after the initial literature review, to guard against the fast-moving small-model landscape becoming stale before study execution; no smaller alternative offered comparable shell-generation evidence within the same CPU-only hardware budget [24]. Newer mixture-of-experts variants require all experts resident in memory regardless of active parameters per token, incompatible with this study's memory budget, and newer general-purpose small models report reasoning benchmarks but no shell-command generation evidence.

b) Execution Pipeline

Once the intent parser produces a shell command, SynapseOS passes it through two safety mechanisms before execution, following the reversibility-and-guardrail framing established for LLM-powered natural language shells [3] and the preview-and-confirm design guideline validated with motor-impaired users of an LLM-mediated assistive-robotics interface [5]. First, the command is classified by reversibility: operations that are irreversible — such as file deletion, system configuration changes, or destructive package operations — trigger a confirmation prompt requiring explicit user approval before execution proceeds. Every executed command is recorded to a persistent undo journal before it runs. Reversible operations are logged for replay-based reversal; a command classified irreversible and then approved is additionally captured by one of four mechanisms — full content backup, hardlink-based trash, git HEAD capture, or permission-metadata capture — so that a command approved in error remains recoverable. Rollback is invoked through a dedicated undo command rather than from within the

conversation. Second, the approved command is dispatched to a bash subprocess. Standard output and standard error are captured and rendered in the conversational surface as the system's response to the original natural language input.

SynapseOS executes only shell-expressible operations. The system's capability boundary is the Linux command-line toolchain: file management, process control, text processing, package management, network configuration, and any application that exposes a command-line interface. GUI-only interactions — clicking buttons in graphical applications, navigating web page elements, or interacting with visual-only interfaces — are explicitly outside scope, in deliberate contrast to broader desktop agent frameworks that additionally orchestrate GUI automation atop the existing session [16]. This boundary is intentional: it constrains the research to a tractable, verifiable claim that the system can be built, evaluated, and defended within the thesis timeline.

c) Recoverability Analysis

The reversibility classification described above is an enumerated list of named command shapes: a proposed command is matched against patterns known to be destructive — *rm*, *dd*, *mkfs*, truncating redirects — and gated if it matches. That mechanism is the baseline this study seeks to replace, and its limitations are structural rather than incidental. It cannot enumerate its domain, because the space of destructive commands is open-ended: extending the list to cover *apt* leaves *snap*, *pip*, *docker rm*, and whatever ships next. It does not compose, because each rule matches a command in isolation while a generated line routinely chains several operations with differing recovery requirements. And it reasons about names rather than effects: *cp a b* is harmless when *b* does not exist and destructive when it does, a distinction no name-matching rule can draw.

This study therefore proposes an algorithm that derives what a command would do from its syntactic structure, and decides recoverability over that derivation. It produces two outputs: a **verdict** — recoverable, recoverable only with prior capture, or unrecoverable — and, for the middle case, a **recovery plan** naming which pre-images must be captured, and by which mechanism, before the command is allowed to run.

The formal object. A command is represented by its *effect set* rather than by its name. An individual *effect* is a triple: a kind, drawn from {*read*, *create*, *write*, *remove*, *metadata*}; a target path, resolved against the working directory in which the command would execute; and the pre-image that would have to be captured for that effect to be reversed. The effect set of a command is the union of the effects of every leaf command within it, taken over the whole of its operator structure.

Extraction. For each leaf command, effects are derived from the command together with its arguments. The derivation is argument-aware because flags change effects and not merely behaviour: *rm -i* differs from *rm -f* in whether removal is certain, *cp -n* cannot overwrite where *cp* can, and *chmod* with and

without `-R` differ in how many paths acquire a metadata effect.

Composition. Effects propagate upward through the operators that join commands into a single line, and the propagation rule differs by operator. Sequencing with `;` accumulates effects unconditionally. Conjunction and disjunction — `&&` and `||` — make later effects conditional on an earlier exit status, so the effect set is a union over the branches that may execute. A pipeline connects standard output to standard input and contributes no filesystem effect of its own, but determines the exit status that gates whatever follows it. Redirection introduces write or truncate effects that appear in no command in the pipeline. Command substitution introduces effects whose targets are computable only at runtime, and `xargs` and `find -exec` multiply a command across an argument set that is not statically known. It is this propagation, rather than the per-command derivation, that an enumerated pattern list has no means of expressing. Parts of a line are analysed in order against an overlay of what earlier parts created or removed, so that in `mkdir out && cp a.txt out/` the copy lands in a directory that does not yet exist, and a path a line creates and then deletes needs no capture because there was nothing there to lose. Loops are analysed once per resolved item with the loop variable bound; `sh -c` strings and transparent wrappers such as `sudo` are analysed as the command they run.

Resolution. Some targets exist only while the command runs: `rm $(ls *.log)`, `find -delete`, `xargs rm`, `git clean -f`, `tar -x`. Rather than declaring these unknowable, the analysis learns them by running a form of the command that cannot change anything — `find` with each action replaced by a print of what the action would have touched, the read-only command that feeds `xargs`, `git clean -n`, `tar -t`, `rsync -n`, the body of a command substitution — and analyses the command again with the concrete targets in place. A resolver is run only if the same analysis proves it has no effects, and where a read-only sandbox is available it runs with the whole filesystem mounted read-only. If it cannot be proven read-only, cannot run, or times out, the targets stay unresolved and the command asks. Resolution is what makes a capture possible for commands whose targets a static reading cannot name.

Verdict and plan. The verdict is a predicate over the composed effect set: a command is recoverable when every effect within it has an available mechanism capable of restoring that effect's pre-image. The recovery plan is a minimum-cost cover of the same set — for each effect, the cheapest mechanism sufficient to restore it, with redundant captures eliminated. Cost is not uniform across mechanisms, which is what makes the selection non-trivial: a hardlink capture is independent of file size where a content copy is proportional to it, a single git object identifier covers an arbitrary number of tracked-file effects at once, and two effects on overlapping paths need be captured only once. The mechanisms themselves — content backup, hardlink-based trash, git HEAD capture, permission-metadata capture — are those already described in the Execution Pipeline; the algorithm selects among them rather than introducing new ones. A rename or move is the one removal that needs no capture, because the same data survives at the destination and undo is moving it back; compression and archiving are not, because the

bytes at the destination are a transformation of the original and moving them back returns the wrong content, so they are captured like any other deletion. The verdict is not where the contribution lies: a list that refuses every command it does not recognise reaches the same verdict on the commands studied. What the plan adds is that the refusal comes with a capture that restores.

Soundness. The analysis is deliberately one-sided. Any construct whose effects cannot be resolved statically — a path arriving from command substitution, an unexpanded variable, a glob that may match a block device — is treated as unrecoverable rather than assumed benign. The two possible errors are not symmetric and are not traded off against each other: over-approximating destructiveness costs the user an unnecessary confirmation prompt, whereas under-approximating it destroys data that cannot be recovered. This asymmetry is why the evaluation in Section 3.1 reports false-negative rate separately rather than folding both directions into a single accuracy figure. Two limits belong to the soundness claim itself. Resolution runs commands before the user has consented, which is safe only because each is proven to have no effects; and the interval between resolving a target and executing the command is a window in which another process can change the filesystem, so a capture is of the state at resolution time and not a guarantee about the state at execution time.

Scope of the model. The effect domain is the filesystem, extended to two kinds of state on the target system that have no pre-image to copy but do have an inverse: installed packages and running or enabled services. For these the analysis does not re-implement the package manager or the service manager. It asks the system's own tools what the command would do — the package manager's simulation mode, the package database, and the service manager's state queries — and derives from the answer the command that puts the state back: a purge for an install, a versioned install for a removal or upgrade, a stop for a start, a disable for an enable. A change is recoverable only if that inverse exists, so removing a package whose old version no repository still offers is unrecoverable, and purging a package also captures its configuration files as removals. Undo runs the inverse, with elevated privilege if the original had it, and prints the command for the user to run if it fails. Not represented are what maintainer scripts and a service's own start-up do beyond the state change, which are arbitrary programs; network mutations and anything on a remote host; and other package managers, which stay unrecoverable unless plainly read-only. A command whose principal effect is one of these is classified unrecoverable by the soundness rule above. This is safe but coarse, and it is stated as a limitation of the model rather than deferred to the results.

The preceding paragraphs are stated formally below, so that the analysis can be checked directly rather than reconstructed from prose.

Algorithm 1 Effect analysis and capture planning for a generated shell command

Input: command c , working directory wd , capture mechanisms M
Output: verdict in $\{\text{RECOVERABLE}, \text{RECOVERABLE_WITH_CAPTURE}, \text{UNRECOVERABLE}\}$
 plan, a set of (mechanism, target) pairs

```

State:    overlay V of paths created or removed by earlier parts of c

function EFFECTS(node, wd, V)
  if node is a leaf command then
    return RULE(name, args, wd, V)           // argument-aware; no rule gives
    {UNKNOWN}
  else if node = (A ; B), (A && B), (A || B), or (A | B) then
    return EFFECTS(A, wd, V)  $\cup$  EFFECTS(B, wd, V)   // B sees what A did,
    through V
  else if node has a redirect target t then
    return EFFECTS(strip_redirect(node), wd, V)  $\cup$  {write-or-create(t)}
  else if node = (for x in W; do body; done) then
    return  $\cup$  over w in EXPAND(W) of EFFECTS(body[x := w], wd, V)
  else if node is sh -c s, or a wrapper such as sudo or env then
    return EFFECTS(inner command, wd, V)
  else if node is find with -delete or -exec, or xargs then
    T  $\leftarrow$  RESOLVE(node)
    if T =  $\perp$  then return {UNRESOLVED}
    return  $\cup$  over t in T of EFFECTS(action[{} := t], wd, V)
  end if
end function

function RESOLVE(node)                       // learns run-time targets, changing
nothing                                       nothing
  d  $\leftarrow$  DRYRUN(node)                     // find with actions replaced by prints; xargs'
producer;                                  // git clean -n, tar -t, rsync -n; command
substitutions
  if EFFECTS(d)  $\neq \emptyset$  or d cannot run then return  $\perp$            // only a proven read-
only d is run
  return the targets d prints
end function

function VERDICT(E)
  if UNRESOLVED  $\in$  E or UNKNOWN  $\in$  E or some effect has no capture then return
UNRECOVERABLE
  if every effect in E is a create or a move then return RECOVERABLE
  return RECOVERABLE_WITH_CAPTURE
end function

function PLAN(E, M)
  P  $\leftarrow \emptyset$ 
  for each path p with a remove, write, or metadata effect in E, in path order
  do
    if p lies under a directory already hardlinked in P then continue
    m*  $\leftarrow$  the cheapest mechanism in M that restores p's effect   // hardlink
    for a removal,                                                    // copy for an
                                                                    overwrite, record for a mode
    P  $\leftarrow$  P  $\cup$  {(m*, p)}
  end for
  return P
end function

```

Algorithm 1. EFFECTS derives and composes the effect set, applying rules to the leaves and resolving wrappers through RESOLVE; RESOLVE learns run-time targets by running only a form of the command proven to have no effects; VERDICT and PLAN decide and cover. UNRESOLVED and UNKNOWN are

the sentinels that give the analysis its one-sided soundness: they can only push the verdict toward UNRECOVERABLE. A move (a removal whose data survives at another path) is the only removal PLAN does not capture.

2.2 Data Pre-processing

Prior to statistical analysis, all data collected during the benchmark evaluation and user study — not the system itself — undergoes the following processing steps:

- **Task normalization:** task descriptions and expected outcomes are standardized across both the OSWorld benchmark and the custom task suite to ensure consistent evaluation criteria
- **Log parsing and event extraction:** raw session logs from SynapseOS telemetry are parsed to extract per-task metrics — completion time, error events, and undo events
- **Metric computation:** task completion time is computed as the elapsed time between task prompt presentation and task completion event; error rate is computed as the ratio of failed or incorrect outcomes to the number of tasks presented — the denominator is the task count rather than the attempt count, so that retrying cannot lower a participant's error rate; each task is scored against a written rubric specifying the observable end state that counts as success, with partial completion, abandonment at the time limit, and facilitator intervention coded as distinct outcomes; and because SynapseOS outcomes are captured by the session logger while native-OS outcomes must be judged from screen recording, all tasks in both conditions are coded from recordings by two researchers independently, with inter-rater agreement reported and disagreements resolved by discussion before analysis; SUS and NASA-TLX scores are computed according to their respective standard scoring formulas
- **Participant data pseudonymization:** all participant records are pseudonymized at the point of extraction, replacing identifying information with participant ID codes prior to any analysis

2.3 Validation

The following validation procedures are applied to ensure the reliability and accuracy of the system and the study instruments:

- **Pilot study:** conducted with five participants prior to the main study to validate the task set difficulty, the questionnaire instruments, and the session procedure; findings from the pilot study are used to revise instruments and procedures before the main data collection begins
- **Intent parsing accuracy:** evaluated using an execution-based scoring methodology [2] — each generated command is executed in a controlled environment and its resulting system state compared against a human-annotated expected outcome for each task utterance, rather than

syntax-level string matching, with ambiguous cases resolved through LLM-assisted functional equivalence assessment at 95% confidence; accuracy is computed as the proportion of first-attempt matches

- **Confirmation policy effectiveness:** evaluated by verifying that all irreversible operations in the task set trigger the confirmation prompt as designed, and that no irreversible operations execute without user approval
- **Undo success rate:** evaluated by executing a representative set of reversible operations and verifying that the undo log correctly restores the prior system state in each case

2.4 Study Session Procedure

Each participant session follows a standardized procedure conducted in the following order:

1. **Briefing:** The session facilitator explains the study purpose, session structure, and participant rights. The participant reads and signs the informed consent form. The pre-study screening questionnaire is administered to confirm group classification.
2. **System familiarization:** For the SynapseOS condition, the participant is given a five-minute orientation to the conversational interface and shown how to enter natural language commands. For the native OS condition, no familiarization is provided — participants use their home environment as they normally would. No task-specific training is provided under either condition.
3. **Task completion:** The participant completes the full task set under the assigned condition. Tasks are presented one at a time via a printed task prompt sheet. The session logger captures start and end times automatically. Screen recording captures all on-screen activity. Participants are not asked to verbalize while tasks are timed. Concurrent think-aloud reliably slows task performance, and here it would slow the two conditions unequally, since narrating while composing a natural-language instruction competes for the same linguistic resources far more than narrating while pointing and clicking. Verbal data is instead collected retrospectively, by replaying the screen recording with the participant during the post-study interview.
4. **Post-condition questionnaire:** Immediately after completing all tasks under a condition, the participant completes the SUS and NASA-TLX questionnaires for that condition.
5. **Condition changeover:** If the participant has not yet completed both conditions, a short break is offered, the participant moves to the other machine (SynapseOS or native OS), and the session returns to step 2.
6. **Post-study interview:** After both conditions are completed, the facilitator conducts a semi-structured interview of approximately 10–15 minutes covering perceived ease of use, trust in the conversational interface, preference between conditions, and open-ended feedback.

7. **Debrief:** The facilitator discloses the full study objectives, answers participant questions, and thanks the participant. Session data is pseudonymized under the participant's ID code at session close; the ID-to-consent mapping is retained for the fourteen-day withdrawal window described in Section 2.5 and destroyed thereafter.

Total estimated session duration per participant: 90–120 minutes.

2.5 Ethical Considerations

This study involves human participants and is subject to ethics review and approval by the Institutional Review Board (IRB) or equivalent ethics committee of Mapúa University – Makati prior to the commencement of participant recruitment. The following ethical principles are observed throughout the study:

- **Informed consent:** All participants receive a written information sheet describing the study purpose, procedures, data collected, and their rights. Participation is entirely voluntary; participants may withdraw at any time without penalty or consequence.
- **Pseudonymization:** Each participant is assigned an ID code at enrollment, and all session logs, questionnaire responses, and transcripts are stored and analyzed under that code alone. Recordings are pseudonymous rather than anonymous: voice is inherently identifying and screen video may capture file names visible on screen, so they cannot be anonymized at the point of collection and are instead governed by the retention rule below.
- **Data minimization:** Only the data types specified in Section 1.1 are collected. No identifying metadata — names, email addresses, or device identifiers — is retained beyond the consent confirmation step.
- **Data security:** All collected data is stored on password-protected institutional infrastructure accessible only to the research team. Interview recordings are deleted after transcription and verification.
- **Confidentiality:** Study findings are reported in aggregate. No individual participant's data is disclosed in any form that could enable identification.
- **Risks and protections:** Participants issue natural-language instructions that cause real commands to execute, including commands capable of deleting or modifying files. That risk is contained by the study environment rather than by restricting what participants may attempt. Both conditions run on dedicated laboratory machines provisioned for the study, holding only synthetic task data and no personal or institutional files; no participant uses a personal device or signs into a personal account at any point; and each machine is restored to a clean baseline image between participants. Participants are informed before consenting that the commands they trigger execute for real on the study machine and that nothing belonging to them is exposed. The foreseeable discomfort is

frustration or a sense of failure while working under observation and timing; participants are told in advance that the interface rather than their competence is under evaluation, and that they may pause or stop without giving a reason.

- **Legal basis and retention:** Personal data in this study is processed under Republic Act No. 10173, the Data Privacy Act of 2012, on the basis of the participant's written consent. Screen recordings and audio are retained only as long as coding requires: both are destroyed once transcription and coding are verified complete, and in no case later than ninety days after the final session. The mapping between ID codes and signed consent forms is stored separately from study data, accessible only to the principal researcher, and destroyed on the same schedule.
- **Right to withdraw:** Participants may discontinue the session at any point, and may request withdrawal of their data for up to fourteen days after their session — the ID-to-consent mapping is retained through that window specifically so that withdrawal remains possible. After it closes the mapping is destroyed and individual records can no longer be located. This limit is stated on the consent form so that it is understood before participation rather than discovered after.

Section 3: System Testing

SynapseOS is evaluated against the datasets and study design established in Section 1. The comparison follows a fixed set of metrics and formulas, a statistical analysis plan for testing significance and effect size, and an accounting of the threats to validity this design addresses.

3.1 Algorithm Evaluation (RQ1)

RQ1 is answered separately from the user study, against a labelled corpus of shell commands rather than through participants. The separation is deliberate and has a practical consequence worth stating: this evaluation depends on no ethics approval, no recruitment, and no participant sessions, and can therefore be completed and reported independently of the study's schedule.

a) Corpus construction

The corpus is generated by a program, not assembled by hand, and is held as three partitions reported separately rather than pooled. The first is NL2Bash [51], which supplies human-written commands and is the generalisation set: candidates are drawn at random from the dataset with a recorded seed, excluding any command used during development. The second is a set of parameterised command templates covering each shell composition operator — sequencing, conditional execution, pipelines, redirects, command substitution, loops, `find -exec`, `xargs`, `sh -c` and other interpreters, and wrappers such as `sudo` and `env` — which exists because naturally occurring corpora under-represent

compound commands, and compound commands are where the algorithm's contribution lies. The third is a set of commands whose effect lies outside the filesystem, described under (b). Commands the prototype generates during the user study are logged and can be labelled afterwards by the same pipeline; they would be a further partition, reported apart, because a set drawn from this prototype's own output is biased toward the shapes it produces.

The corpus is sized against the classes the analysis has to separate, not against a single overall total. Commands are drawn until a fixed number carry each ground-truth label, so that the label the false-negative test needs — a command that loses something — is present in quantity whatever NL2Bash's own composition, which is mostly read-only. The target is on the order of 750 commands: about 450 from NL2Bash, of which 150 are losses; about 250 from templates; and about 80 external. This is a coverage-based target rather than a power-computed one: a classical power analysis needs an assumed effect size, and the quantity of interest, the false-negative rate, is expected to be low for both the baseline and the algorithm, the regime in which an assumed effect size would be a guess. What the corpus must give is enough losses that a Wilson interval on a proportion near zero remains informative, and 150 does.

A second stratification is by the shape of the command, assigned by the generator from the command's syntax: a single named command; a chain; one whose targets are computed while it runs; a redirect; one whose effect hides inside an interpreter or a tool the rule table does not name; and one whose effect lies outside the filesystem. The claims under test concern the composed, run-time, and hidden shapes, and a corpus dominated by plain named commands would measure the strength of a list and not the contribution of the analysis, so results are reported per shape as well as in aggregate.

b) Ground truth

Ground truth is observed, not judged. Each executable command is given a fixture — the files and directories that exist when it runs — and is run once inside a bubblewrap sandbox: the rest of the filesystem read-only, no network, empty standard input, a hard time limit, and only the fixture writable. The tree is recorded before and after, and the label follows from the difference. A command is *recoverable* when nothing that existed was lost: paths may have been created, and a path that disappeared but whose very same file now sits elsewhere was moved, not lost. It is *recoverable only if a pre-image is captured* when something that existed was removed, overwritten, replaced by a different kind of object, retargeted, or given different permission bits or an owner. Modification times are ignored, since touching a file loses nothing a user could want back. Commands that do not finish cleanly — a non-zero exit, a timeout, an attempt to write outside the fixture — are excluded and counted, never labelled by guess.

Commands whose effect the filesystem cannot show — on another machine, on the mount table, on another process, on a device, on an account or the firewall — cannot be labelled this way and are never executed. They form the external partition and are labelled *unrecoverable* by construction. The rule that

assigns a command to it is deliberately narrow, naming only tools that change such state, so that a read-only network query is not counted as a loss the system failed to catch. It is a partition rule and not a source of the other two labels.

Ground truth is defined relative to a stated filesystem state. A command's effects depend on what exists: *rm *.tmp* deletes nothing where there are no such files, and *cp a b* overwrites only if *b* is there. A label given with no state is a label about an imagined directory, and an analysis that reads the real state cannot be scored against it. Every corpus item therefore carries its fixture, and the analysis is run against that same fixture. The fixture is built from the command's text before the analysis is consulted: the files and directories the command names are created, a glob becomes a few matching files, and whether the destination of a copy, move, or redirect already exists is chosen by the seed, with the other choice tried if the first does not run. Fixtures are never written from the analysis's output.

The generator shares no code with the analysis and never consults it, so agreement between the two is evidence and not circularity. It is deterministic for a given seed, and it records the number of candidates drawn and the reasons for each exclusion beside the corpus. There is no human labeller, and so no labeller disagreement to measure; the threat that takes its place is the validity of the oracle itself, addressed in (e).

c) Baselines

Three baselines bound the comparison. The first is the enumerated-pattern classifier already implemented in the prototype, which matches commands against a hand-maintained list of known destructive shapes; it is the honest comparison because it is what the system does today, not a weakened construction. The second is the same list turned around to fail closed: every command in the line, including those inside substitutions, loops, and wrappers, must be a known read-only form or the line asks. It resolves no targets and plans no capture, and it is the strongest simple alternative, because it removes the enumerated list's silent failures on commands nobody named. The upper bound on safety is a full pre-emptive copy of the working directory before every command, which recovers everything and is unusable in practice — it establishes what the minimal-plan computation must justify itself against on cost.

d) Metrics

Table 3.3. Metrics for the recoverability algorithm, and what each is for.

Metric	What it measures
Silent loss	Commands labelled recoverable-with-capture or unrecoverable that run with no confirmation and no capture. Primary. The failure that destroys a user's data without their being asked

Metric	What it measures
Capture coverage	Of commands labelled recoverable-with-capture, the share for which the system captured before running and the capture restores the prior state, verified by executing the command in a sandbox, undoing it, and comparing the tree. Primary. A confirmation that is answered yes and then cannot be undone is counted against this, not credited as safe
False-negative rate	Commands judged recoverable that are not. Reported on its own rather than folded into an accuracy figure , because this is the error that destroys a user's data and it is not interchangeable with the error below
False-positive rate	Commands judged unrecoverable that are recoverable. The cost is unnecessary friction, which matters because habituation to unnecessary confirmations is a documented failure mode of gate designs
Recovery success rate	Of commands whose plan claimed to cover them, the proportion whose prior state was actually restored — verified by executing the command in a sandboxed copy, applying the plan, and diffing against the pre-state
Storage and time overhead	Cost of the computed plan against the full-snapshot baseline. This is what "minimal" has to earn
Composition coverage	Proportion of compound commands resolved without falling back to an unrecoverable verdict — the measure most directly tied to what the enumerated-pattern approach cannot do at all

e) Threats specific to this evaluation

Four are anticipated. Corpus bias is addressed by partitioning, as described above. Oracle validity replaces label subjectivity: the labels are observed, so they cannot be argued with, but they are facts about one command on one small fixture, run for a few seconds, seeing the working tree and not repository internals. A loss that appears only on a differently shaped tree, or after the time limit, is labelled by what happened here. The oracle's own behaviour is pinned by unit tests over commands whose outcome is known, its exclusions are reported, and the bias they introduce — toward commands that run on a small fixture, which excludes most NL2Bash candidates that name paths or hosts the fixture cannot supply — is stated and not corrected. Third, the algorithm and the labels are produced by the same research team, which creates a risk that the labels reflect the algorithm's assumptions; the generator is independent of the analysis by construction, and the corpus is fixed by a recorded seed and hash before the analysis is run on it. A fourth threat follows from the same fact: the analysis rests on a table of what each known command does, written by the same team, and a table written with the corpus in view measures the cases its author anticipated. The corpus is therefore generated twice from different seeds. Disagreements on the first are used to correct the analysis; the primary result is reported on the second, drawn after the rule table is frozen, with the table's version recorded beside the result.

f) Statistical comparison against baseline

The algorithm and the pattern-list baseline are both scored against the same labelled corpus, which makes this a paired rather than an independent-samples comparison: for every command, each method's verdict is either correct or incorrect against the ground-truth label, and the two methods' correctness on that one command is not independent of each other. Significance is therefore tested with McNemar's exact test on the resulting 2×2 table of discordant pairs — commands the algorithm gets right and the baseline gets wrong, and the reverse — at $\alpha = 0.05$, rather than with a two-sample test that would treat the corpus as though two different sets of commands had been drawn.

Because false-negative rate is the safety metric (Table 3.3) and is reported on its own rather than folded into overall accuracy, it receives its own comparison: McNemar's test restricted to the subset of the corpus that carries a loss (labelled recoverable-only-with-capture, or external), where a false negative is defined, plus a 95% Wilson confidence interval on each method's false-negative rate individually. The Wilson interval is used rather than the normal approximation because false-negative rate is expected to be low for both methods, and the normal approximation is unreliable near the boundary of a proportion — the exact regime this metric sits in.

A significance test states only whether the two methods differ, not by how much, so each comparison is paired with an effect size — the paired-data equivalent of the Cohen's *d* reported for the user study (§3.5(d)). The primary measure is the **risk difference**: the algorithm's accuracy minus the baseline's, with a Newcombe-adjusted 95% confidence interval built for paired proportions, reported for overall verdict accuracy and again for the false-negative-rate comparison specifically. The **odds ratio** from the same 2×2 discordant-pairs table is reported alongside it, since it is the effect-size convention most directly tied to McNemar's test itself and lets the result be compared against other paired-classifier studies that report it in preference to a risk difference.

With three baselines and two primary metrics, the comparisons that carry a claim are the analysis against each of the first two baselines on silent loss and on capture coverage. Four tests share one family, so significance is judged with a Holm correction at a family-wise $\alpha = 0.05$, and the uncorrected values are reported beside the corrected ones. The full-snapshot baseline is a bound on cost and safety, not a comparison of accuracy, and is reported descriptively.

g) Preliminary evidence

The procedure above has been run once, on the prototype, so that the proposal rests on a measurement and not only on a design. It is preliminary and is not the study's result: the analysis was corrected against a development corpus of 778 commands (seed 20260922) before the second corpus was drawn, and the second, held-out corpus of 772 commands (seed 20260923; 450 from NL2Bash, 242 from templates, 80 external; 316 losses among the executable items) was run once, after the rule table was frozen. On that corpus the enumerated-pattern classifier let 155 of the 396 commands that lose something

run with no confirmation and no capture (39%, 95% Wilson interval 34–44%). The fail-closed list let none through silently, but it asked on every one of the 316 executable losses and captured nothing for any of them, and it asked on 111 of the 376 commands that lose nothing (30%). The effect analysis let 1 of the 396 through silently (0.3%, interval under 1.5%), planned a capture before running for 278 of the 316 executable losses (88%; whether those captures restore the prior state is the separate execute-undo-diff check of Table 3.3, not measured here), and asked on 41 of the 376 harmless commands under the strict policy (11%) and 30 under the capture policy (8%). By shape, the run-time targets that the list misses are where the analysis differs most: of 155 losses whose targets are computed while the command runs, the list let 47 through silently and the analysis 1.

Two caveats limit what this shows. The remaining commands the analysis asked on without capturing are mostly ones it cannot see into — an interpreter or an unknown program (*perl -e*, *python -c*, a script by path), a variable with no value in the fixture, a privileged command whose targets it declines to resolve, and *shred*, which destroys content by design — and it asks on these by construction. And the one silent loss is a fixture artefact that reproduces: a directory holding a file whose name contains a backslash, matched by an escaped glob. It is reported, not corrected, because the corpus is fixed. The confirmatory analysis in (f) — McNemar's test, effect sizes, and the Holm correction — is applied to a corpus drawn after the design is final, not to this one.

3.2 Evaluation Dataset

The system is evaluated against two datasets: the OSWorld benchmark task suite [17] for direct comparison with results in the existing literature, and the custom cross-platform task suite described in Sections 1.1 and 1.5. The user study evaluation involves 40 participants (20 novice, 20 power users); the baseline condition is each participant's primary operating system — Windows 11, macOS, or Linux with GNOME desktop — on a dedicated baseline machine.

3.3 Evaluation Profile

The study employs a within-subjects design with two conditions:

- **Condition A:** SynapseOS (conversational interface layer on the Debian 13 evaluation machine)
- **Condition B:** Participant's primary operating system (Windows 11, macOS, or Linux with GNOME desktop, as determined by pre-study screening, on the dedicated baseline machine)

All 40 participants complete both conditions. Condition order is counterbalanced: 20 participants complete Condition A first, and 20 participants complete Condition B first. This counterbalancing controls for order effects and task familiarity.

The two participant groups — novice users and power users — are analyzed both in aggregate and separately to determine whether the effect of the interface condition differs across experience levels. Additionally, primary OS (Windows, macOS, Linux) is recorded as a secondary grouping variable to determine whether results vary across participant backgrounds, and prior conversational AI exposure (frequency of use) is recorded as a covariate to determine whether familiarity with LLM-based tools correlates with SynapseOS performance independent of OS proficiency. The primary analysis is the pooled within-subjects comparison across all 40 participants; per-OS subgroup analyses and AI-exposure correlations are treated as exploratory given the smaller per-group sample sizes they entail.

3.4 Criteria and Formula

The following metrics and formulas are used to evaluate SynapseOS against the baseline:

a) Task Completion Time

Mean task completion time is computed per condition per participant group. Inferential testing and effect size reporting follow the procedures described in Section 3.5.

b) Error Rate

Error rate is computed as the proportion of task attempts resulting in failure or incorrect outcome:

$$\text{Error Rate} = (\text{Failed Tasks} + \text{Incorrect Outcomes}) / \text{Total Task Attempts}$$

Error events are further categorized by type: intent parsing errors (incorrect bash command generated), execution errors (bash subprocess failure), and recovery failures (unsuccessful undo or retry). The same paired statistical test is applied to error rate differences between conditions.

c) User Satisfaction

SUS scores are computed per the standard scoring procedure [40] to produce a score on a 0–100 scale; a score above 68 is conventionally treated as above-average usability [42], and the instrument's validity and interpretation across a large sample of studies are established by Bangor and colleagues [43]. NASA-TLX scores are computed as the unweighted mean of the six subscale ratings [41] to produce a composite workload score per participant per condition. Because the central claim of this study concerns access rather than speed, workload and task success are treated as primary outcomes alongside completion time, not as secondary measures.

Qualitative interview data is analyzed using thematic analysis to identify recurring themes in participant perceptions of both conditions.

d) Statistical Significance and Effect Size

All primary comparisons between conditions are tested at $\alpha = 0.05$. Effect size is reported using Cohen's d , with thresholds of 0.2 (small), 0.5 (medium), and 0.8 (large). Sample size is justified by an a priori power analysis for a paired-samples comparison at $\alpha = 0.05$, two-tailed, computed against the noncentral t distribution. Detecting a medium effect ($d = 0.5$) at 80% power requires 34 participants; the 40-participant sample provides approximately 87% power at that effect size and detects effects down to $d = 0.45$, giving margin for attrition and for exclusions arising from the screening rule in Section 1.4. Analysed alone, each 20-participant subgroup detects effects of $d = 0.66$ or larger. The condition $\times$ group interaction — the test corresponding most directly to the study's central claim — remains the least-powered comparison in the design, since an interaction of a given size requires substantially more participants than a main effect of the same size; it is therefore reported with effect sizes and confidence intervals as the primary descriptive finding, and is not treated as a confirmatory significance test.

3.5 Data Analysis Plan

Quantitative analysis proceeds in three stages. First, descriptive statistics — means, standard deviations, and 95% confidence intervals — are computed for all primary metrics (task completion time, error rate, SUS score, NASA-TLX score) per condition and per participant group. Second, the Shapiro-Wilk test is applied to assess normality of each distribution before selecting the appropriate inferential test: the paired-samples t -test for normally distributed data, or the Wilcoxon signed-rank test for non-normally distributed data. Third, effect size is computed for all primary comparisons regardless of statistical significance: Cohen's d where the paired-samples t -test is applied, and the matched-pairs rank-biserial correlation where the Wilcoxon signed-rank test is applied.

Subgroup analysis is conducted separately for the novice and power user groups to determine whether the effect of the interface condition differs across populations. The interaction between interface condition and user group is examined to assess whether SynapseOS produces differential effects depending on technical background.

Qualitative data from post-study interviews is analyzed using reflexive thematic analysis following the six-phase procedure described by Braun and Clarke [23]: familiarization with the data, initial code generation, theme search, theme review, theme definition, and write-up. Two members of the research team independently code a random subset of transcripts; inter-rater agreement is assessed and disagreements are resolved through discussion before finalizing the codebook. Themes are reported alongside representative participant quotations.

3.6 Threats to Validity

The following threats to validity are identified and the mitigations applied in this study are described.

Internal validity:

- **Learning effect and asymmetric transfer:** Participants may perform better on the second condition through familiarity rather than interface quality, and counterbalancing alone does not resolve this: it redistributes carryover but only cancels it when transfer is symmetric, which cannot be assumed here, since having solved a task once confers a different advantage in each direction. *Mitigation:* parallel task forms matched on category, difficulty, and step count are crossed with condition order in a 2×2 design, so no participant meets the same task twice; task order within a form is randomized; and condition order is entered as a between-subjects factor in the analysis, with the condition × order interaction inspected so that residual transfer is measured rather than presumed absent.
- **Demand characteristics:** Participants may behave differently from natural use because they know they are being observed. *Mitigation:* participants are instructed to complete tasks as they naturally would; the specific hypotheses of the study are withheld until the post-study debrief to reduce social desirability bias.
- **Fatigue:** Completing the full task set twice in a single session may degrade performance in the second condition independent of interface quality. *Mitigation:* a short break is offered between conditions and total session length is bounded to approximately 120 minutes.
- **Fallback reliance:** GUI mode provides a participant-accessible fallback to the underlying XFCE session if SynapseOS becomes unresponsive or a participant wishes to stop using it mid-task; unrestricted, this would let a participant's own choice substitute for the interface under evaluation, confounding the within-subjects comparison. *Mitigation:* every fallback invocation is logged with participant ID, task ID, and timestamp; any task during which it is invoked is excluded from the primary completion-time and error-rate comparison for that task rather than counted as a SynapseOS success, and fallback-invocation rate is reported separately as an exploratory reliability metric.

External validity:

- **Sample generalizability:** The study uses a university-recruited sample of 40 participants across three primary OS backgrounds (Windows, macOS, Linux), which limits direct generalization to the broader population of computer users. Findings should be interpreted as indicative rather than definitive; replication with larger and more diverse samples is recommended as a direction for future work.

- **Task generalizability:** The evaluation is bounded by OSWorld coverage and the custom task suite developed for this study. Computing workflows outside this coverage are not evaluated, and results may not extend to all possible use cases.
 - **Platform specificity:** SynapseOS is evaluated on a single hardware and software configuration — a dedicated machine running SynapseOS in GUI mode (XFCE desktop environment, X11 session) on a Debian 13 (Trixie) base. Performance characteristics on different hardware, display configurations, or Linux distributions may differ from the reported results.
-

Notes

1. Terminal User Interface: a text-based interactive interface that runs in a command-line terminal. TUI mode requires no graphical display server or desktop environment and can be accessed remotely over SSH.
2. The SynapseOS prototype uses Qwen2.5-Coder-3B-Instruct, a code-specialized open-weights model selected for its NL2Bash accuracy. See Section 2.1a for selection rationale and benchmark evidence.
3. An inference server loads a language model into memory and processes query requests from the application layer. Ollama is an open-source tool for running language models locally on consumer hardware without cloud connectivity.
4. Open-weights models make their trained parameters publicly available for local download and use; users run inference on their own hardware without routing queries to an external service provider.
5. Quantization reduces the numerical precision of model weights to lower memory requirements. At 4-bit precision, a model that would otherwise require dedicated GPU memory can run on a standard CPU with modest RAM.
6. Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning method that introduces a small number of trainable weight matrices while keeping the base model frozen, enabling task-specific adaptation on consumer hardware without full retraining.
7. NL2Bash is a benchmark dataset pairing natural language command descriptions with their corresponding bash shell commands, used to evaluate a model's accuracy in translating natural language into executable commands.
8. Command-Line Interface mode: a single non-interactive invocation, with no session persisting between separate calls, that runs a bounded multi-step loop toward one natural-language request — each step proposed, classified for reversibility, executed (immediately if reversible, after confirmation if not) through the same gate as the other interface modes, and its result fed back so the model can propose the next step or signal completion — up to a fixed step limit, before the invocation exits. Distinct from TUI mode's persistent interactive chat session; intended for scripting, automation, and one-off remote commands.

REFERENCES

- [1] Wilensky, R., Chin, D. N., Luria, M., Martin, J., Mayfield, J., & Wu, D. (1988). The Berkeley Unix Consultant Project. *Computational Linguistics*, 14(4), 35–84. <https://aclanthology.org/J88-4003/>
- [2] Westenfelder, F., Hemberg, E., Tulla, M., Moskal, S., O'Reilly, U.-M., & Chiricescu, S. (2025). LLM-Supported Natural Language to Bash Translation. *arXiv:2502.06858*. <https://arxiv.org/abs/2502.06858>
- [3] Gyawali, B. R., Achalla, S., Kallas, K., & Kumar, S. (2025). NaSh: Guardrails for an LLM-Powered Natural Language Shell. *arXiv:2506.13028*. <https://arxiv.org/abs/2506.13028>
- [4] van Dam, H. G. W. (2025). A Multimodal GUI Architecture for Interfacing with LLM-Based Conversational Assistants. *arXiv:2510.06223*. <https://arxiv.org/abs/2510.06223>
- [5] Padmanabha, A., Yuan, J., Gupta, J., Karachiwalla, Z., Majidi, C., Admoni, H., & Erickson, Z. (2024). VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM. <https://doi.org/10.1145/3654777.3676401>
- [6] Mehendale, S., & Walishetti, A. (2024). DexAssist: A Voice-Enabled Dual-LLM Framework for Accessible Web Navigation. *arXiv:2411.12214*. <https://arxiv.org/abs/2411.12214>
- [7] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., & Su, Y. (2023). Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. <https://arxiv.org/abs/2306.06070>
- [8] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2024). WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of ICLR 2024*. <https://arxiv.org/abs/2307.13854>
- [9] Zheng, B., Gou, B., Kil, J., Sun, H., & Su, Y. (2024). GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of ICML 2024*. <https://arxiv.org/abs/2401.01614>
- [10] Rawles, C., Li, A., Rodriguez, D., Riva, O., & Lillicrap, T. (2023). Android in the Wild: A Large-Scale Dataset for Android Device Control. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. <https://arxiv.org/abs/2307.10088>
- [11] Zhang, C., Yang, Z., Liu, J., Han, Y., Chen, X., Huang, Z., Fu, B., & Yu, G. (2023). AppAgent: Multimodal Agents as Smartphone Users. *arXiv:2312.13771*. <https://arxiv.org/abs/2312.13771>

- [12] Wang, J., Xu, H., Ye, J., Yan, M., Shen, W., Zhang, J., Huang, F., & Sang, J. (2024). Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *arXiv:2401.16158*. <https://arxiv.org/abs/2401.16158>
- [13] Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Zhang, Y., Li, J., Xu, B., Dong, Y., Ding, M., & Tang, J. (2024). CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of CVPR 2024*. <https://arxiv.org/abs/2312.08914>
- [14] Niu, R., Li, J., Wang, S., Fu, Y., Hu, X., Leng, X., Kong, H., Chang, Y., & Wang, Q. (2024). ScreenAgent: A Vision Language Model-driven Computer Control Agent. *arXiv:2402.07945*. <https://arxiv.org/abs/2402.07945>
- [15] Hui, T., Zhang, Y., Jiang, B., Sun, J., Cheng, F., Lin, X., & Yang, Y. (2025). WinClick: GUI Grounding with Multimodal Large Language Models. *arXiv:2503.04730*. <https://arxiv.org/abs/2503.04730>
- [16] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). UFO²: The Desktop AgentOS. *arXiv:2504.14603*. <https://arxiv.org/abs/2504.14603>
- [17] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., & Yu, T. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. <https://arxiv.org/abs/2404.07972>
- [18] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). Large Language Model-Brained GUI Agents: A Survey. *arXiv:2411.18279*. <https://arxiv.org/abs/2411.18279>
- [19] Li, Y., He, J., Zhou, X., Zhang, Y., & Baldridge, J. (2020). Mapping Natural Language Instructions to Mobile UI Action Sequences. In *Proceedings of ACL 2020*, 8198–8210. <https://doi.org/10.18653/v1/2020.acl-main.729>
- [20] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of ICLR 2023*. <https://arxiv.org/abs/2210.03629>
- [21] Han, W., Zeng, Z., Huang, J., Jiang, S., et al. (2025). GUIRoboTron-Speech: Towards Automated GUI Agents Based on Speech Instructions. *arXiv:2506.11127*. <https://arxiv.org/abs/2506.11127>
- [22] Park, J., Tang, P., Das, S., Appalaraju, S., Singh, K. Y., Manmatha, R., & Ghadar, S. (2025). R-VLM: Region-aware Vision Language Model for Precise GUI Grounding. In *Findings of the Association for Computational Linguistics: ACL 2025*, 9669–9685. <https://doi.org/10.18653/v1/2025.findings-acl.501>
- [23] Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), 77–101. <https://doi.org/10.1191/1478088706qp0630a>

- [24] Jacobs, J., Lapon, J., & Naessens, V. (2026). Local LLMs for NL2Bash: A Large-Scale Open-Source Model Evaluation for Bash Command Generation. *NDSS Workshop on LLM Assisted Security and Trust Exploration (LAST-X 2026)*. <https://www.ndss-symposium.org/wp-content/uploads/lastx2026-49.pdf>
- [25] Lu, J., Zhang, Z., Yang, F., Zhang, J., Wang, L., Du, C., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). AXIS: Efficient Human-Agent-Computer Interaction with API-First LLM-Based Agents. *arXiv:2409.17140*. <https://arxiv.org/abs/2409.17140>
- [26] Yuan, M., Zhou, Z., Xiong, X., et al. (2026). OSWorld 2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks. *arXiv:2606.29537*. <https://arxiv.org/abs/2606.29537>
- [27] Zhou, X., Zhang, S., Zhao, Y., Wei, J., Song, T., Cohan, A., & Zhao, C. (2026). GUI vs. CLI: Execution Bottlenecks in Screen-Only and Skill-Mediated Computer-Use Agents. *arXiv:2606.24551*. <https://arxiv.org/abs/2606.24551>
- [28] Liu, R., Zhe, T., Wang, D., Yao, Z., Liu, K., Fu, Y., Liu, H., & Pei, J. (2026). AgentOS: From Application Silos to a Natural Language-Driven Data Ecosystem. *arXiv:2603.08938*. <https://arxiv.org/abs/2603.08938>
- [29] Alibaba Cloud. (2026). ANOLISA: A server-side operating layer for AI agent workloads. Retrieved September 2026. <https://github.com/alibaba/anolisa>
- [30] eva AG. (2026). Warmwind OS. Retrieved September 2026. <https://warmwind.space/>
- [31] Shneiderman, B. (1983). Direct manipulation: A step beyond programming languages. *Computer*, 16(8), 57–69. <https://doi.org/10.1109/MC.1983.1654471>
- [32] Norman, D. A. (1986). Cognitive engineering. In D. A. Norman & S. W. Draper (Eds.), *User Centered System Design: New Perspectives on Human-Computer Interaction* (pp. 31–61). Lawrence Erlbaum Associates.
- [33] Hutchins, E. L., Hollan, J. D., & Norman, D. A. (1985). Direct manipulation interfaces. *Human-Computer Interaction*, 1(4), 311–338. https://doi.org/10.1207/s15327051hci0104_2
- [34] Shneiderman, B., & Maes, P. (1997). Direct manipulation vs. interface agents. *interactions*, 4(6), 42–61. <https://doi.org/10.1145/267505.267514>
- [35] Shneiderman, B. (2000). The limits of speech recognition. *Communications of the ACM*, 43(9), 63–65. <https://doi.org/10.1145/348941.348990>
- [36] Luger, E., & Sellen, A. (2016). "Like having a really bad PA": The gulf between user expectation and experience of conversational agents. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems (CHI '16)*, 5286–5297. <https://doi.org/10.1145/2858036.2858288>
- [37] Clark, L., Pantidi, N., Cooney, O., Doyle, P. R., Garaialde, D., Edwards, J., Spillane, B., Gilmartin, E., Murad, C., Munteanu, C., Wade, V., & Cowan, B. R. (2019). What makes a good conversation? Challenges in designing truly conversational agents. In *Proceedings of the 2019 CHI Conference on*

- Human Factors in Computing Systems (CHI '19)*, Paper 475, 1–12.
<https://doi.org/10.1145/3290605.3300705>
- [38] Norman, D. A. (1981). The trouble with UNIX: The user interface is horrid. *Datamation*, 27(12), 139–150.
- [39] Whiteside, J., Jones, S., Levy, P. S., & Wixon, D. (1985). User performance with command, menu, and iconic interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '85)*, 185–191. <https://doi.org/10.1145/317456.317490>
- [40] Brooke, J. (1996). SUS: A "quick and dirty" usability scale. In P. W. Jordan, B. Thomas, I. L. McClelland, & B. Weerdmeester (Eds.), *Usability Evaluation in Industry* (pp. 189–194). Taylor & Francis.
- [41] Hart, S. G., & Staveland, L. E. (1988). Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In P. A. Hancock & N. Meshkati (Eds.), *Human Mental Workload* (Advances in Psychology, Vol. 52, pp. 139–183). North-Holland.
[https://doi.org/10.1016/S0166-4115\(08\)62386-9](https://doi.org/10.1016/S0166-4115(08)62386-9)
- [42] Sauro, J., & Lewis, J. R. (2016). *Quantifying the User Experience: Practical Statistics for User Research* (2nd ed.). Morgan Kaufmann.
- [43] Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human–Computer Interaction*, 24(6), 574–594.
<https://doi.org/10.1080/10447310802205776>
- [44] Creswell, J. W., & Plano Clark, V. L. (2018). *Designing and Conducting Mixed Methods Research* (3rd ed.). SAGE Publications.
- [45] Cousot, P., & Cousot, R. (1977). Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. *Conference Record of the Sixth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '77)*, 238–252. <https://www.di.ens.fr/~cousot/publications.www/CousotCousot-POPL-77-ACM-p238--252-1977.pdf>
- [46] Jouvelot, P., & Gifford, D. K. (1991). Algebraic reconstruction of types and effects. *Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '91)*, 303–310.
https://www.researchgate.net/publication/2391518_Algebraic_Reconstruction_of_Types_and_Effects
- [47] Talpin, J.-P., & Jouvelot, P. (1994). The type and effect discipline. *Information and Computation*, 111(2), 245–296. <https://www.sciencedirect.com/science/article/pii/S0890540184710467>
- [48] Mazurak, K., & Zdanczewicz, S. (2007). ABASH: Finding bugs in bash scripts. *Proceedings of the 2007 Workshop on Programming Languages and Analysis for Security (PLAS '07)*.
<https://doi.org/10.1145/1255329.1255347>

- [49] Greenberg, M., & Blatt, A. J. (2020). Executable formal semantics for the POSIX shell. *Proceedings of the ACM on Programming Languages*, 4(POPL), Article 43. <https://doi.org/10.1145/3371111>
- [50] Becker, B., Jeannerod, N., Marché, C., Régis-Gianas, Y., Sighireanu, M., & Treinen, R. (2022). The CoLiS platform for the analysis of maintainer scripts in Debian software packages. *International Journal on Software Tools for Technology Transfer*, 24(5), 717–733. <https://doi.org/10.1007/s10009-022-00671-1>
- [51] Lin, X. V., Wang, C., Zettlemoyer, L., & Ernst, M. D. (2018). NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*. European Language Resources Association. <https://aclanthology.org/L18-1491/>

Appendices

This section contains the study instruments administered to participants and the procedure governing their administration. Wording, difficulty calibration, and task-form equivalence are validated during the pilot study described in Section 2.3 and revised before the main study begins; the instruments are reproduced here in the form submitted for ethics review.

Appendix A: System Usability Scale (SUS)

Administered immediately after each condition, referring to the interface just used. Items are the standard ten-item instrument of Brooke [40], with the word “system” replaced throughout by “interface” so that participants rate the interaction surface rather than the machine. Each item is answered on a five-point scale from 1 (strongly disagree) to 5 (strongly agree).

Participant ID: _____ **Condition:** ☐ SynapseOS ☐ Own operating system **Date:**

#	Statement	1	2	3	4	5
1	I think that I would like to use this interface frequently.	☐	☐	☐	☐	☐
2	I found the interface unnecessarily complex.	☐	☐	☐	☐	☐
3	I thought the interface was easy to use.	☐	☐	☐	☐	☐
4	I think that I would need the support of a technical person to be able to use this interface.	☐	☐	☐	☐	☐
5	I found the various functions in this interface were well integrated.	☐	☐	☐	☐	☐
6	I thought there was too much inconsistency in this interface.	☐	☐	☐	☐	☐
7	I would imagine that most people would learn to use this interface very quickly.	☐	☐	☐	☐	☐
8	I found the interface very cumbersome to use.	☐	☐	☐	☐	☐
9	I felt very confident using the interface.	☐	☐	☐	☐	☐
10	I needed to learn a lot of things before I could get going with this interface.	☐	☐	☐	☐	☐

Scoring (not shown to participants). For odd-numbered items, subtract 1 from the response. For even-numbered items, subtract the response from 5. Sum the ten adjusted scores and multiply by 2.5, giving a value from 0 to 100. This value is not a percentage; interpretation follows Sauro and Lewis [42] and Bangor and colleagues [43].

Appendix B: NASA Task Load Index (NASA-TLX)

Administered immediately after each condition, referring to the set of tasks just completed. Six subscales are rated on a twenty-one-point scale presented as a line divided into equal intervals and scored 0 to 100 in steps of five. This study uses the unweighted variant (Raw TLX): the pairwise weighting procedure of the original instrument [41] is omitted, and the composite workload score is the arithmetic mean of the six subscale ratings. Note that the Performance subscale is anchored in the opposite direction to the others and is reverse-scored before averaging.

Participant ID: _____ **Condition:** ☐ SynapseOS ☐ Own operating system **Date:**

Subscale	Question	Low anchor	Rating (0–100, steps of 5)	High anchor
Mental Demand	How mentally demanding was the task?	Very low	_____	Very high
Physical Demand	How physically demanding was the task?	Very low	_____	Very high
Temporal Demand	How hurried or rushed was the pace of the task?	Very low	_____	Very high
Performance	How successful were you in accomplishing what you were asked to do?	Perfect	_____	Failure
Effort	How hard did you have to work to accomplish your level of performance?	Very low	_____	Very high
Frustration	How insecure, discouraged, irritated, stressed and annoyed were you?	Very low	_____	Very high

Appendix C: Custom Cross-Platform Task Suite

The suite comprises sixteen tasks in two parts. Twelve *core* tasks span the four categories defined in Section 1.5 at three difficulty levels, and constitute the primary comparison. Four *boundary* tasks deliberately fall outside what a shell-mediated interface can express; they are administered and scored but reported separately from the primary comparison, because their purpose is to characterise the capability boundary honestly rather than to contribute to the between-condition contrast. Omitting them would leave the suite composed only of work the command line does well, which would make any favourable result difficult to defend.

Every task exists in two parallel forms, matched on category, difficulty, and number of required steps, and differing only in target. No participant encounters the same form twice; form and condition order are crossed as described in Section 2.4. All prompts are phrased as human goals in plain language,

containing no shell syntax, no command names, and no interface-specific vocabulary, so that they are equally answerable through natural language on SynapseOS and through native tools on the baseline machine.

Tasks T7–T9 require identical provisioning on all study machines — the same designated applications present or absent, and the two test applications (*SampleApp*, *DemoTool*) installed — so that the prompt means the same thing under both conditions and on all three baseline operating systems; this provisioning is part of the facilitator checklist in Appendix G. Task T9 is classified irreversible and is included specifically to exercise the confirmation gate; T11 mutates file content and is included to exercise the undo journal. Difficulty is assigned by the number of distinct operations required and the specificity of the target, and is validated during the pilot study described in Section 2.3.

Table C.1. Custom cross-platform task suite — core tasks.

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
T1	File search & organization	Simple	Find every PDF in the Documents folder and gather them into one folder named <i>PDFs</i> .	Find every image in the Downloads folder and gather them into one folder named <i>Images</i> .	All matching files present in the target folder; no matching file left behind or lost.
T2	File search & organization	Moderate	Show the ten largest files anywhere under the home folder.	Show the ten oldest files anywhere under the home folder.	The correct ten files identified, largest/oldest first.
T3	File search & organization	Complex	Find files modified in the last seven days and copy them into a folder named <i>recent-work</i> , preserving their folder structure.	Find files larger than 10 MB modified this month and copy them into <i>large-recent</i> , preserving their folder structure.	Correct file set copied; directory structure preserved; originals untouched.
T4	System & process monitoring	Simple	Identify the program currently using the most memory.	Identify the program currently using the most CPU.	Correct process named.
T5	System & process monitoring	Moderate	Report free disk space, and which folder in the home directory occupies the most of it.	Report free memory, and the three programs consuming the most of it.	Both figures correct at time of measurement.
T6	System & process monitoring	Complex	A process is consuming excessive CPU. Identify it and stop it.	A process is consuming excessive memory. Identify it and stop it.	Correct process identified and terminated; no unrelated process stopped.

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
T7	Application & package management	Simple	Determine whether VLC is installed, and report its version.	Determine whether GIMP is installed, and report its version.	Correct installation status and version reported.
T8	Application & package management	Moderate	Report how many applications are installed on this machine.	Report which application on this machine was installed most recently.	Correct figure reported, verified against the machine's provisioning record.
T9	Application & package management	Complex	A provisioned test application named <i>SampleApp</i> is installed. Uninstall it completely, including the configuration files it created.	A provisioned test application named <i>DemoTool</i> is installed. Uninstall it completely, including the configuration files it created.	Application and its configuration removed; nothing else removed. Irreversible — exercises the confirmation gate.
T10	Text & data processing	Simple	Report how many lines <i>report.csv</i> contains.	Report how many lines <i>sales.csv</i> contains.	Correct line count.
T11	Text & data processing	Moderate	In <i>notes.txt</i> , replace every occurrence of “draft” with “final”.	In <i>log.txt</i> , replace every occurrence of “WARN” with “WARNING”.	All occurrences replaced; no other content altered. Content-mutating — exercises undo.
T12	Text & data processing	Complex	From <i>sales.csv</i> , report total revenue per region, highest first.	From <i>orders.csv</i> , report order count per customer, highest first.	Correct aggregation and ordering.

Table C.2. Boundary tasks — outside shell expressibility, reported separately.

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
C1	Visual editing	Moderate	Crop the supplied photograph to the person on the left.	Rotate the supplied photograph so the horizon is level.	Result judged correct by the facilitator against a reference image.
C2	Visual comparison	Simple	State which of the two supplied images is brighter.	State which of the two supplied images is more blurred.	Correct image identified.
C3	Document formatting	Moderate	Make the heading of the supplied document bold and centred.	Change the supplied document's body font to 14 point.	Formatting applied correctly, verified in the saved file.

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
C4	Window management	Simple	Arrange the four open windows so all are visible simultaneously.	Minimise every open window except the text editor.	End state matches the requested arrangement.

Appendix D: Pre-Study Screening Questionnaire

Completed before enrolment and used to classify each volunteer as a novice or command-line-fluent participant, and to record the covariates described in Section 1.4. The form records a participant ID code only; the volunteer’s name appears solely on the consent form (Appendix F), which is stored separately from all study data. Screening data belonging to volunteers who are not enrolled is destroyed within seven days of the screening session.

Participant ID: _____ **Date:** _____

Part 1 — Background (self-report)

1. Are you 18 years of age or older? ☐ Yes ☐ No (*a “No” response ends the screening; the study enrolls adults only*)
2. Approximately how many years have you used a computer regularly? _____
3. Which operating system do you use most? ☐ Windows ☐ macOS ☐ Linux
4. Roughly how many hours per day do you use that operating system? _____ (*four or more hours is required for enrolment*)
5. How would you describe your proficiency with that operating system? ☐ Basic ☐ Intermediate ☐ Advanced
6. How would you describe your familiarity with a command line or terminal on any platform? ☐ None ☐ Basic ☐ Intermediate ☐ Advanced
7. How often do you use conversational AI tools such as chat assistants or AI coding assistants? ☐ Never ☐ Occasionally ☐ Regularly ☐ Daily
8. If you use such tools, which ones? _____

Part 2 — Behavioural screener

Three short tasks performed unaided on the volunteer’s own primary operating system using its native tools, one per system-task category used in the main study. The screener exists because self-reported proficiency is an unreliable classifier: fluency with an operating system’s own tools is the property being measured, not confidence about it. Each task is scored pass or fail by the facilitator against the stated time limit and completion criterion. The volunteer is told that the screener determines which group they join and that neither outcome affects eligibility for payment or participation.

#	Task	Time limit	Pass criterion	Result
S1	Locate the files most recently modified in your home or user folder and move them into a new folder.	3 minutes	Correct files identified and relocated unaided.	☐ Pass ☐ Fail
S2	Using your operating system's own system monitor, identify the application currently consuming the most memory.	3 minutes	Correct application named unaided.	☐ Pass ☐ Fail
S3	Determine whether a named application is installed on this machine and report its version.	3 minutes	Correct status and version reported unaided.	☐ Pass ☐ Fail

Classification (facilitator use). A volunteer is classified *command-line fluent* if they report advanced proficiency on their primary operating system, report at least basic command-line familiarity, and pass all three screener tasks. A volunteer is classified *novice* if they report basic proficiency, report no command-line familiarity, and fail all three screener tasks. Any volunteer whose self-report and screener results disagree, or who falls between these definitions, is not enrolled. This rule retains only unambiguous cases and therefore samples the extremes of the proficiency range deliberately; the consequence for generalisation is stated in Section 3.6.

Classification: ☐ Novice ☐ Command-line fluent ☐ Not enrolled **Assigned task form:** ☐ A first ☐ B first **Assigned condition order:** ☐ SynapseOS first ☐ Own OS first

Appendix E: Participant Information Sheet

Study title. SynapseOS: Designing and Evaluating a Conversational Interface Layer for Personal Computing

Who is running this study. This research is conducted by undergraduate researchers in the Department of Computer Science, Mapúa University – Makati, in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science. It has been reviewed and approved by the University's Institutional Review Board. Contact details for the research team and for the ethics committee are given at the end of this sheet.

Why we are doing this. Most of what a computer can do is reachable only by people who know how to write commands for it. We have built a system that lets a person say what they want in ordinary language — for example, “find the files I changed last week and put them in one folder” — and carries it out. We want to find out whether that is genuinely easier to use than the desktop you already use, and whether it makes more difference for people who have never used a command line.

What you would do. You would attend one session of about 90 to 120 minutes. You would complete a set of everyday computer tasks twice: once using our system, and once using the operating system you normally use, on a computer we provide. Which one you start with is decided in advance and is

not a test of you. Afterwards you would fill in two short questionnaires about each interface and take part in a 10 to 15 minute conversation about your experience, during which we may replay the screen recording and ask what you were thinking at particular moments.

What we record. We record what happens on the screen during the tasks, audio of the closing conversation, your questionnaire answers, and the times and outcomes of the tasks. We do not record your name against any of this. Everything is filed under a code.

What could go wrong, and what protects you. The instructions you give will cause real commands to run, and some commands can delete or change files. You will be working on a laboratory computer prepared for this study, holding only made-up practice files. You will not use your own device and will not sign in to any of your own accounts, so none of your own data is involved at any point. Each machine is reset to a clean state between participants. Some people find working under observation and a timer uncomfortable, or feel frustrated when a task does not go well. We are evaluating the interface, not you, and you may pause or stop at any moment without giving a reason.

Benefits. There is no direct benefit to you. The findings are intended to inform how conversational interfaces to computers are designed, particularly for people without technical training.

How your information is handled. Your data is processed under Republic Act No. 10173, the Data Privacy Act of 2012, on the basis of your written consent. It is stored under a code rather than your name, on password-protected university infrastructure that only the research team can reach. Screen recordings and audio cannot be made fully anonymous — a voice identifies a person, and a screen can show file names — so we keep them only as long as we need them for analysis. They are destroyed once transcription and coding are complete, and in any case no later than ninety days after the last session. The list linking your code to your signed consent form is kept separately and destroyed on the same schedule.

Withdrawing. You may stop the session at any point, without explanation and without any consequence. You may also ask us to delete your data for up to fourteen days afterwards; we keep the code-to-consent list for exactly that period so that we can find and remove your records if you ask. After fourteen days that list is destroyed and we can no longer identify which records are yours, so withdrawal is not possible beyond that point.

How the results are used. Results are reported in aggregate in a thesis and possibly in academic publications. We may quote short remarks you make in the closing conversation. Quotations are attributed only to a participant code and a group label, never to your name, and you can decline to be quoted while still taking part in everything else.

Questions or concerns. For questions about the study, contact the research team at [research team contact address]. For concerns about how the study is being conducted, or about your rights as a participant, contact the Institutional Review Board of Mapúa University – Makati at [ethics committee contact address], independently of the research team.

Appendix F: Informed Consent Form

Study title. SynapseOS: Designing and Evaluating a Conversational Interface Layer for Personal Computing

Please read the Participant Information Sheet (Appendix E) before completing this form, and tick each box to confirm your agreement. You may ask questions at any time before signing.

Statement
☐ I have read and understood the Participant Information Sheet, and I have had the opportunity to ask questions.
☐ I am 18 years of age or older.
☐ I understand that my participation is voluntary, and that I may stop at any time without giving a reason and without any consequence.
☐ I understand that the instructions I give will cause real commands to run on a laboratory computer, that this computer holds only practice files prepared for the study, and that none of my own devices, accounts, or files are involved.
☐ I understand that my data is stored under a code, that the list linking that code to my name is kept separately, and that I may request deletion of my data for up to fourteen days after my session, after which it can no longer be identified.
☐ I understand that some information about the study's specific expectations is withheld until the end of the session, and that it will be explained to me fully before I leave.
☐ I agree to take part in this study.

Separate permissions. You may take part in the study without agreeing to these. Please tick only those you accept.

Statement
☐ I agree to the screen being recorded while I complete the tasks.
☐ I agree to the closing conversation being audio-recorded, and understand the recording is destroyed once it has been transcribed and checked.
☐ I agree that short anonymous quotations from what I say may appear in the thesis or in academic publications, identified only by a participant code.

Participant name (printed): _____ Signature: _____
 Date: _____

Researcher obtaining consent (printed): _____ Signature: _____
 Date: _____

Filing note: this is the only study document bearing the participant's name. It is stored separately from all study data, and the code-to-name list derived from it is destroyed on the schedule stated in Section 2.5.

Appendix G: Session Facilitator Checklist

Followed identically for every participant so that session conduct does not vary across the sample.

Items marked † must be completed before the participant arrives.

Before the session †

1. Restore both machines to the clean baseline image; confirm no data remains from the previous participant.
2. Boot the baseline machine into the operating system matching the participant's screening record (Windows, macOS, or Linux with GNOME).
3. Provision the task environment identically on both machines: practice file set in place, the applications referenced by tasks T7–T9 present or absent as specified, and the test applications *SampleApp* and *DemoTool* installed.
4. On the SynapseOS machine, confirm the inference server is running and the model is loaded, and run one throwaway task to confirm the system responds.
5. Confirm screen recording works on both machines and has free storage for the full session.
6. Lay out the printed task prompt sheet for the assigned form, the two questionnaire sets, the information sheet, and the consent form.
7. Record which interface mode SynapseOS is running in for this session, per the readiness checkpoint in the System Environment Specifications.

During the session

1. Give the information sheet, answer questions, obtain signed consent. File the consent form separately from all study materials.
2. Give the five-minute orientation to the assigned first condition. Use the same wording for every participant; do not demonstrate any task from the task set.
3. Start screen recording. Present tasks one at a time in the randomised order for this participant. Do not prompt, hint, or answer questions about how to accomplish a task.
4. Do not ask the participant to think aloud during timed tasks.
5. Record the outcome of each task and note any facilitator intervention, with the reason.
6. Administer SUS and NASA-TLX immediately on completing the condition, before any discussion of it.
7. Offer a short break. Move to the second machine and repeat from step 3 with the other task form.
8. Conduct the closing conversation, replaying portions of the screen recording to prompt recall. Ask what the participant was thinking, not whether they liked the system.
9. Debrief: explain the study's expectations in full, answer questions, and thank the participant.

After the session

1. Transfer all recordings and logs to protected storage under the participant code; delete local copies from both machines.
2. Log any fallback invocation, with participant code, task, and timestamp.
3. Note any deviation from this checklist, however small, for reporting with the results.
4. Restore both machines to the clean baseline image.
5. Diarise the participant's fourteen-day withdrawal deadline.